

OSI Model — Layers 1–7 (Condensed)

Addressing Scope (applies across L2/L3/L4 — stated once)

MAC (L2) → local link only, rebuilt every hop | **IP (L3)** → end-to-end, never changes | **Port (L4)** → process-specific, end-to-end

Encapsulation Chain (stated once — see per-layer PDU name only below)

Data → [L4 header]=Segment → [L3 header]=Packet → [L2 header+trailer]=Frame → [L1]=Bits

L1 — Physical

Moves raw bits only. No frames/packets/addresses/decisions — hence "dumb layer."

- **Job:** encode bits → signal (voltage/light/radio) and back; defines connectors, cabling, voltage, clocking, bit rate, topology
- **Cables:** UTP (unshielded, Cat5e/6) · STP (shielded) · Coax (legacy 10BASE-2/5) · Fibre (immune to EMI, long-haul)
- **RJ45:** T568A vs T568B — only pairs 2↔3 (orange/green) swapped; crossover = A one end, B other end
- **Encoding:** NRZ (steady voltage, clock-drift prone) vs Manchester (mid-bit transition, self-clocking)
- **Serial vs Parallel:** serial = one wire, long distance, networking | parallel = many wires, short distance, skew/EMI issues
- **Async vs Sync:** async = start/stop bits, no shared clock (RS-232) | sync = shared/embedded clock, continuous (modern links)
- **Bandwidth vs Throughput:** bandwidth = theoretical max | throughput = actual (always ≤ bandwidth, overhead/noise reduce it)
- **Duplex:** Simplex (one-way, TV) · Half (turns, walkie-talkie/hub) · Full (both ways, switch port)
- **Topologies:** Mesh (redundant) · Star (common, central switch) · Ring (one break = failure) · Bus (legacy shared cable)

Devices: Repeater (regen signal, extends distance) → Hub (multi-port repeater, 1 collision domain; active=boosts, passive=just wires) → Cables/Connectors (RJ-45, BNC, LC, SC, DB-9) → NIC (L1+L2) → Modem (digital↔analog) → CSU/DSU (router↔WAN, clocking) → Transceiver (media conversion)

Protocols/Standards: Ethernet 802.3, Wi-Fi 802.11, Bluetooth 802.15, USB, RS-232, DSL, SONET/SDH, V.35, DOCSIS, T1/E1 (1.544/2.048 Mbps)

L2 — Data Link

Hop-to-hop frame delivery between two **directly connected** devices only. IP packet (L3) stays same end-to-end; **frame is destroyed & rebuilt at every router hop.**

PC1→R1 (frame1) | R1→R2 (frame2, new MACs) | R2→PC2 (frame3, new MACs)

IP packet inside: unchanged throughout – only the frame wrapper changes

Sublayers: LLC (802.2 — interfaces L3, flow/error control, multiplexing) · MAC (802.3/802.11 — framing, addressing, medium access)

Frame: Preamble | Dst MAC | Src MAC | Type/Len | Data(IP pkt) | FCS

- MAC = 48-bit, AA:BB:CC:DD:EE:FF, first 3 bytes = OUI (vendor), last 3 = device ID, local-link only
- Fixed-size frames (ATM, 53B) vs variable-size (Ethernet, HDLC — delimiter flags)

Functions:

Function	Mechanism
Framing	headers/trailers around L3 packet
Flow control	Stop-and-Wait (1 frame, wait ACK) · Sliding Window (multiple before ACK)
Error control	FCS/CRC (most common) · Parity bit · ACK+retransmit · Sequence numbers
Media access	CSMA/CD (wired, listen+detect collision) · CSMA/CA (Wi-Fi, avoid via backoff) · Token Passing (Token Ring) · TDMA (cellular/WAN)

Protocols: Ethernet 802.3, Wi-Fi 802.11, PPP, HDLC (Cisco default serial), Frame Relay, ATM (53B cells), ARP (L2/L3 boundary — resolves IP→MAC), STP 802.1D (loop prevention), VLAN 802.1Q, PPPoE, SDLC, SLIP

Devices: Switch (MAC table, per-port collision domain) · Bridge (2-port predecessor to switch) · NIC · Wireless AP (bridges 802.11↔802.3)

L3 — Network

End-to-end packet delivery **across** networks via logical (IP) addressing. IP src/dst never change; only L2 frame rebuilds per hop; L4 payload untouched at routers.

- **Direct delivery:** same network, no router (192.168.1.10→.20) | **Indirect:** different networks, via router(s)
- **Logical addressing:** IPv4 (32-bit) / IPv6 (128-bit); Network ID + Host ID (192.168.1.10/24 → net .0, host .10)
- **Packetization:** L4 segment + IP header = packet
- **Routing:** Static (manual) · Dynamic (RIP/OSPF/EIGRP, auto-exchange) · Default (0.0.0.0/0 catch-all)
 - Routing = path determination (control plane) vs Forwarding = actual send (data plane)
- **Fragmentation:** triggered when packet > MTU (Ethernet default 1500B); Identification/Fragment Offset/MF flag; **reassemble only at final destination**
- **NAT:** Static (1:1 fixed) · Dynamic (pool) · PAT (many:1 via ports — most common, home routers)

IPv4 header key fields: Version · IHL · ToS/DSCP · Total Length · Identification/Flags/Fragment Offset (fragmentation) · **TTL** (dec per hop, drop@0, prevents loops) · Protocol (TCP=6, UDP=17, ICMP=1) · Header Checksum · Src/Dst IP

Protocols: IPv4, IPv6, ICMP (ping/traceroute/errors), ICMPv6+NDP, ARP (boundary, resolves IP→MAC), RARP (obsolete), NAT, IPSec, MPLS (label-based, ISP traffic engineering)

Routing protocols:

Protocol	Type	Metric	Max hops	Use
RIP	Distance-vector	Hop count	15 (16=unreachable)	Small nets
OSPF	Link-state	Cost (bandwidth)	Unlimited	Enterprise; uses Dijkstra SPF , builds LSDB
EIGRP	Adv. distance-vector (Cisco)	BW+Delay+Load+Reliability	Unlimited	Cisco-only
BGP	Path-vector	AS path attrs	Unlimited	Internet backbone

Routing vs Flooding: routing = best path via table, efficient, low BW | flooding = all paths, inefficient, high BW (e.g. ARP broadcast)

Devices: Router (primary) · L3 Switch (routes VLANs via SVI) · Multilayer Switch (L2+L3) · Firewall (IP/protocol filtering)

L4 — Transport

Process-to-process delivery via **ports**. L3 gets packet to the right *host*; L4 gets it to the right *application*.

Socket = IP:Port (e.g. 192.168.1.10:80)

Ports: 0–1023 well-known (HTTP80, HTTPS443, FTP21, SSH22, DNS53, SMTP25) | 1024–49151 registered (MySQL3306, RDP3389) | 49152–65535 ephemeral

- **Segmentation:** message → tagged segments (seq #) → reassembled in order; detects lost/duplicate/out-of-order
- **Connection control:** TCP (connection-oriented, reliable) vs UDP (connectionless, fast/unreliable)
 - 3-way handshake: **SYN** → **SYN-ACK** → **ACK**
 - 4-way termination: **FIN** → **ACK** → **FIN** → **ACK**
- **Flow control (end-to-end, not hop-to-hop):** TCP sliding window — receiver advertises window size; shrinks/grows with receiver speed
- **Error control (end-to-end):** Checksum, ACK, Retransmission (on RTO timeout), Sequence numbers

TCP vs UDP vs SCTP:

	TCP	UDP	SCTP
Connection	3-way handshake	Connectionless	4-way handshake

	TCP	UDP	SCTP
Reliability	ACK+retransmit	None	Selective ACK
Ordering	Guaranteed	None	Both modes
Header	20–60B	8B (no seq/ACK/window — bare minimum for speed)	12B+chunks
Multihoming	No	No	Yes
Proto #	6	17	132
Use	HTTP,FTP,SSH,SMTP	DNS,DHCP,VoIP,SNMP	Telephony (SS7/IP)

Devices: Host OS TCP/IP stack · Stateful firewall (port+state inspection) · Load balancer (L4 port/protocol) · Proxy (terminates/re-establishes TCP)

L5 — Session

Dialog controller: establishes/manages/synchronizes/terminates sessions between application processes. Pure software.

- **Lifecycle:** Establishment (negotiate params, select duplex) → Data transfer (checkpoints) → Termination
- **Dialog control:** Half-duplex (turn-taking, token-based) vs Full-duplex (simultaneous)
- **Synchronization (checkpoints)** — key differentiator: without checkpoints, a failed 900MB/1GB transfer restarts from 0; with checkpoints, resume from last major/minor sync point (only resend since then)
- **Session mgmt:** Session ID, re-sync after failure, multiplexing (multiple L4 services share one connection). Auth/authz coordinated here but **enforced at L7** (OAuth, HTTP Basic, LDAP)

Protocols: RPC, NetBIOS, SIP (session setup=L5, signaling=L7), RTCP, PPTP/L2TP (VPN tunneling), PAP, H.245

Devices: none dedicated (software) — app servers, proxy servers, stateful firewalls, SBCs, VPN gateways manage sessions

L6 — Presentation

"Translator" — makes data readable across systems regardless of internal format. Pure software.

- **Translation:** EBCDIC↔ASCII, ASCII↔Unicode, big↔little-endian, JPEG/PNG↔bitmap; uses **ASN.1** as common description language; handles serialization (JSON/XML→bytes)
- **Encryption/decryption:** confidentiality via encode/decode; SSL/TLS commonly taught here, but in practice TLS runs over TCP(L4) serving L7 apps — **TCP/IP model puts TLS at Application layer**
- **Compression:** Lossless (ZIP/GZIP/LZW — text, no loss) vs Lossy (JPEG images, MPEG/H.264 video, MP3/AAC audio — acceptable for media, not text)

Protocols: SSL/TLS, JPEG, MPEG, GIF/PNG, ASCII/EBCDIC/Unicode, XDR, MIME (Base64), ASN.1

Devices: none (software) — browsers (TLS+parsing), media players (decompression), email clients (MIME), OpenSSL/Bouncy Castle

L7 — Application

Entry point for user data into the OSI stack. Note: the *application itself* (Chrome) isn't L7 — the *protocols* enabling its network use are.

Service	Protocol	Port
Web	HTTP / HTTPS	80 / 443
Remote admin	SSH / Telnet	22 / 23
Mgmt	SNMP	161/162
Time sync	NTP	123
IP assignment	DHCP	67/68

Devices: End-user hosts, web servers, DNS servers, mail servers (SMTP/POP3/IMAP), DHCP servers, L7 firewall (DPI), L7 load balancer (URL/cookie/header routing)

Encapsulation/Decapsulation Summary

Layer	Adds (send)	PDU	Strips (receive)
L7/L6/L5	format, compress, encrypt	Data	decrypt, decompress, deliver to app
L4	TCP/UDP header (ports, seq, checksum)	Segment	check ports/seq, strip, reassemble
L3	IP header (src/dst IP, TTL, protocol)	Packet	check IP, strip
L2	MAC header + FCS trailer	Frame	check MAC+CRC, strip
L1	frame→bits→transmit	Bits	bits→reconstruct frame

OSI Layers — Real-World Mapping

Which PC component handles each layer:

Layer	Component	Example
7. Application	Application software	Chrome, WhatsApp, Outlook
6. Presentation	Application / libraries	TLS encryption, JSON, image encoding
5. Session	Application / OS libraries	Login session, connection session
4. Transport	OS network stack	TCP, UDP
3. Network	OS network stack	IPv4, IPv6, ICMP
2. Data Link	NIC driver + NIC	Ethernet frame, MAC address
1. Physical	NIC hardware / PHY	Electrical signals, Wi-Fi radio

Security controls per layer (ties into CDAC DITISS/security focus):

Layer	Main Security	Examples
7. Application	Application security	WAF, authN/authZ, secure coding, antivirus
6. Presentation	Encryption / data protection	TLS/SSL, encryption, certificates
5. Session	Session security	Session timeout, secure cookies, MFA, session tokens
4. Transport	Port/connection security	Firewall rules, TCP/UDP filtering, TLS
3. Network	IP/routing security	Firewall, ACL, IPsec, VPN, IDS/IPS
2. Data Link	LAN/switch security	VLAN, Port Security, 802.1X, DHCP Snooping, ARP protection
1. Physical	Physical protection	Locked server room, CCTV, access cards, cable protection

1. What Is an IP Address?

An **IP (Internet Protocol) address** is a unique number given to every device on a network so it can send and receive data. It does two jobs: 1. **Identifies the device** 2. **Locates the device**

IP Address = Network Portion(which network?) + Host Portion (which device on it?)

- **IPv4** → 32 bits long, written as **192.168.1.10**
- **IPv6** → 128 bits long, written as **2001:db8::1**

1.1 How Traffic Gets Delivered: Unicast, Broadcast, Multicast, Anycast

Every packet on a network travels using one of four delivery styles:

Type	Goes To	Works In	In Plain Words
Unicast	One specific device	IPv4 & IPv6	One sender, one receiver — normal web browsing traffic.
Broadcast	Every device on the local network	IPv4 only	One sender, everyone on the segment gets it (e.g. 255.255.255.255). IPv6 has no broadcast at all — multicast does this job instead.
Multicast	Only devices that joined a group	IPv4 & IPv6	One sender, only "subscribed" devices receive it. IPv4 range: 224.0.0.0–239.255.255.255 . IPv6 range: ff00::/8 .
Anycast	The <i>nearest</i> device in a group	Mainly IPv6 (limited IPv4 use)	One sender, delivered to whichever member of the group is closest by routing distance.

One-liners to remember:

- Unicast = 1 → 1
- Broadcast = 1 → everyone (IPv4 only)
- Multicast = 1 → subscribed group (both IPv4 & IPv6)
- Anycast = 1 → nearest member of a group (mostly IPv6)

2. IPv4 Address Classes

IPv4 addresses are split into **5 classes**, identified by the value of the **first octet**.

127.x.x.x is carved out of Class A and reserved for **loopback** testing.

Class	First Octet Range	Default Mask	# Networks	# Hosts / Network	Format
A	1 – 126	255.0.0.0	126 (2^7-2)	16,777,214 ($2^{24}-2$)	N.H.H.H
B	128 – 191	255.255.0.0	16,384 (2^{14})	65,534 ($2^{16}-2$)	N.N.H.H
C	192 – 223	255.255.255.0	2,097,152 (2^{21})	254 (2^8-2)	N.N.N.H
D	224 – 239	— (Multicast)	—	—	—
E	240 – 255	— (Experimental)	—	—	—

Key points:

- "N" = network bits, "H" = host bits.
- Class D is for **multicast only** — it has no host/network split and no subnet mask.
- Class E is **reserved for research** — also no subnet mask.
- Class-based addressing (A/B/C/D/E) is largely historical today — real networks use **CIDR** (Section 4), which ignores class boundaries. Classes are still worth knowing for exams and for reading legacy documentation.

3. Subnetting

Subnetting means splitting one large network into smaller networks by "borrowing" bits from the host portion to extend the network portion.

Default subnet masks (by class):

Class	Default Subnet Mask
A	255.0.0.0
B	255.255.0.0
C	255.255.255.0

Why subnet at all?

-  Cuts down broadcast traffic (smaller broadcast domains)

- ☒ Gets around the fixed host-count limits of a single classful network
- ☒ Lets you expose only part of a network for security/segmentation (e.g. a guest Wi-Fi subnet)

Core formulas:

```
Usable subnets = 2^(subnet bits)
Usable hosts    = 2^(host bits) - 2
Block size      = 256 - (subnet mask octet value)
```

Older textbooks subtract 2 from the subnet count for an "all-zeros" and "all-ones" subnet. Modern routers (RFC 1878) allow both, so in practice **all 2ⁿ subnets are usable** — but some exam boards still test the old "-2" rule, so know both.

3.1 Class C Example — Mask 255.255.255.224 (/27)

Last octet in binary: 1110 0000 (3 subnet bits, 5 host bits)

- Subnet bits = 3 → up to **8 subnets** (6 if using the older -2 convention)
- Host bits = 5 → $2^5 - 2 = 30$ **usable hosts per subnet**
- Block size = $256 - 224 = 32$

Subnet	Broadcast	Valid Host Range
32	63	33–62
64	95	65–94
96	127	97–126
128	159	129–158
160	191	161–190
192	223	193–222

3.2 Class B Example — Mask 255.255.240.0 (/20)

- 4 bits borrowed → $2^4 = 16$ **subnets** (14 under the old convention)
- 12 host bits → $2^{12} - 2 = 4,094$ **hosts per subnet**
- Block size = $256 - 240 = 16$

Subnet	First Host	Last Host	Broadcast
.16.0	.16.1	.31.254	.31.255
.32.0	.32.1	.47.254	.47.255
.48.0	.48.1	.63.254	.63.255
.64.0	.64.1	.79.254	.79.255

3.3 Class A Example — Mask 255.240.0.0 (/12)

- 4 bits borrowed → $2^4 = \mathbf{16 \text{ subnets}}$ (14 under the old convention)
- 20 host bits → $2^{20} - 2 = \mathbf{1,048,574 \text{ hosts per subnet}}$
- Block size = $256 - 240 = \mathbf{16}$

First subnet:

```
Subnet      : 10.0.0.0
Broadcast   : 10.15.255.255
Hosts       : 10.0.0.1 - 10.15.255.254
```

Last subnet:

```
Subnet      : 10.240.0.0
Broadcast   : 10.255.255.255
Hosts       : 10.240.0.1 - 10.255.255.254
```

4. CIDR (Classless Inter-Domain Routing)

CIDR replaces the rigid Class A/B/C system with a simple **slash notation** (/n) that states exactly how many bits, counting from the left, form the network portion. This lets a network be *any* size, not just a class-sized one.

192.168.1.0/24

└─ "/24" = first 24 bits = NETWORK part
 = last 8 bits = HOST part
 = subnet mask 255.255.255.0

4.1 Why CIDR Matters

- ☒ No more wasted addresses (a Class C used to force exactly 254 hosts even if you only needed 10)
- ☒ Enables **route summarization** — many small networks advertised as one big block, shrinking routing tables
- ☒ Is the foundation that makes **VLSM** possible (Section 5)

4.2 Prefix Length ↔ Subnet Mask ↔ Hosts (IPv4 Quick Reference)

CIDR (/n)	Subnet Mask	# Host Bits	Usable Hosts ($2^n - 2$)
/24	255.255.255.0	8	254
/25	255.255.255.128	7	126
/26	255.255.255.192	6	62
/27	255.255.255.224	5	30
/28	255.255.255.240	4	14

CIDR (/n)	Subnet Mask	# Host Bits	Usable Hosts ($2^n - 2$)
/29	255.255.255.248	3	6
/30	255.255.255.252	2	2 (point-to-point links)
/32	255.255.255.255	0	1 (single host route)

Formula:

Usable Hosts = $2^{(32 - n)} - 2$ where n = CIDR prefix length

4.3 Route Summarization (Supernetting)

CIDR also lets you go the *other* direction — combine several small networks into one larger advertised block:

```

192.168.0.0/24
192.168.1.0/24
192.168.2.0/24
192.168.3.0/24
└─┬─> Summarized as 192.168.0.0/22
    (covers .0.0 - .3.255, 1024 addresses)

```

- **Shorter prefix** (fewer network bits) → bigger block, more hosts, fewer routes to advertise.
- **Longer prefix** (more network bits) → smaller block, fewer hosts, more granular control.

4.4 CIDR in IPv6

Same /n idea, e.g. `2001:db8::/32`. IPv6 almost always standardizes on a **/64** network prefix, leaving the remaining 64 bits for the device's own identifier (see Section 7).

One-liners:

- CIDR = slash notation that states the network-bit count, replacing rigid classes.
- Shorter prefix = bigger block. Longer prefix = smaller block.
- CIDR supports both subnetting (splitting) and summarization (combining).

5. VLSM (Variable Length Subnet Mask)

VLSM, also called **classless addressing**, lets you use *different* mask lengths for different subnets carved from the same network — long masks (small blocks) for small departments, short masks (big blocks) for large ones. It requires a **classless routing protocol** (e.g. OSPF, EIGRP) to work, since those protocols carry the mask along with each route.

Why use VLSM instead of one fixed subnet size for everyone? Because department sizes vary. A fixed subnet size forces you to either waste addresses on small departments or run out of addresses for big ones. VLSM sizes each subnet to fit its actual need, minimizing waste.

Golden rule: always allocate the largest requirement first, then carve the remaining space for smaller ones.

Worked Example — 192.168.1.0/24

Requirements (sorted largest → smallest):

Department	Hosts Needed
Sales	100
Purchase	50
Accounts	25
Management	5

Steps:

1. Sort requirements from biggest to smallest.
2. Give the biggest requirement the smallest mask that still fits it (most hosts).
3. Move to the next unused block of address space and repeat for the next requirement.

Dept	Network	Prefix	Mask	Usable Hosts
Sales	192.168.1.0	/25	255.255.255.128	126
Purchase	192.168.1.128	/26	255.255.255.192	62
Accounts	192.168.1.192	/27	255.255.255.224	30
Management	192.168.1.224	/29	255.255.255.248	6

```
192.168.1.0/24
├─ 192.168.1.0/25   → Sales      (126 hosts)
├─ 192.168.1.128/26 → Purchase  (62 hosts)
├─ 192.168.1.192/27 → Accounts  (30 hosts)
└─ 192.168.1.224/29 → Management (6 hosts)
    └─ 192.168.1.232 - 255 → still unused, kept for future growth
```

6. Wildcard Mask & Wildcard IP

A **wildcard mask** is the *inverse* of a subnet mask. It's used in **ACLs (Access Control Lists)** and **OSPF network statements**, and it flips the logic of a normal subnet mask:

```
Subnet Mask   : 1 = network bit (must match)  0 = host bit (varies)
Wildcard Mask : 0 = must match                 1 = don't care (any value OK)

Wildcard Mask = 255.255.255.255 - Subnet Mask
```

6.1 Quick Reference

CIDR (/n)	Subnet Mask	Wildcard Mask
/24	255.255.255.0	0.0.0.255
/25	255.255.255.128	0.0.0.127
/26	255.255.255.192	0.0.0.63
/27	255.255.255.224	0.0.0.31
/28	255.255.255.240	0.0.0.15
/29	255.255.255.248	0.0.0.7
/30	255.255.255.252	0.0.0.3
/32	255.255.255.255	0.0.0.0
/0	0.0.0.0	255.255.255.255

Worked example:

```

Subnet Mask   : 255.255.255. 0   → 11111111.11111111.11111111.00000000
Wildcard Mask :   0.   0.   0.255 → 00000000.00000000.00000000.11111111
                (each octet: 255 - subnet-octet = wildcard-octet)

```

6.2 Where Wildcard IPs Are Used

Use Case	Example Syntax	Meaning
ACL permit/deny	<code>access-list 10 permit 192.168.1.0 0.0.0.255</code>	Matches any host 192.168.1.0–192.168.1.255
OSPF network statement	<code>network 10.0.0.0 0.0.0.255 area 0</code>	Enables OSPF on interfaces in 10.0.0.0/24
Match a single host	<code>access-list 10 permit 192.168.1.5 0.0.0.0</code>	Matches only 192.168.1.5 — every bit must match
Match anything	<code>access-list 10 permit 0.0.0.0 255.255.255.255</code>	Matches any address — shortcut keyword: any

One-liners:

- Wildcard mask = inverse of subnet mask.
- **0** bit = must match exactly | **1** bit = don't care.
- All-**0.0.0.0** wildcard = match exactly one host.
- All-**255.255.255.255** wildcard = match every address.
- Wildcard masks configure **ACLs and OSPF**, not an interface's actual IP (that still uses a normal subnet mask).

7. IPv6 Addressing

- **128 bits** long (vs IPv4's 32 bits) — written as 8 groups of hex digits.
- Identifies a network interface and enables routing, exactly like IPv4 does, just with a vastly bigger address pool.
- **No broadcast in IPv6** — the all-nodes multicast group `ff02::1` covers most of that role, though protocol-specific multicast groups are preferred in practice.

7.1 IPv6 Address Types

Type	Delivered To	Notes
Unicast	One specific interface	Standard one-to-one delivery.
Multicast	All interfaces in a group	Replaces IPv4 broadcast; devices join a group to receive it.
Anycast	The nearest interface in a group	Same address <i>format</i> as unicast — the only difference is that the address is assigned to multiple devices, and routing sends traffic to whichever one is closest. A common real-world use is DNS root servers: many physical servers share one anycast address, and each client reaches the nearest one.

7.2 Writing IPv6 Addresses

Full form — 8 groups of 4 hex digits ("hextets"), separated by `::`:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

Rule 1 — drop leading zeros in each group (keep at least one digit):

```
2001:db8:85a3:0:0:8a2e:370:7334
```

Rule 2 — replace ONE run of consecutive all-zero groups with `::` (only once per address, to avoid ambiguity about how many groups it stands for):

```
2001:db8:85a3::8a2e:370:7334
```

Special addresses:

Address	Meaning
<code>::1</code>	Loopback (full: <code>0:0:0:0:0:0:0:1</code>)
<code>::</code>	Unspecified address (full: <code>0:0:0:0:0:0:0:0</code>)

7.3 Structure of an IPv6 Address: Network ID + Interface ID

A typical /64 IPv6 address splits cleanly into two 64-bit halves:

```
3A:2B1C : 0000:0000 : 0009:0101:0000:007C /64
└──────────┬──────────┘
Network ID  Interface ID
(first 64 bits) (last 64 bits)
```

Part	Bits	Also Called	Purpose
Network ID	First 64	Network Prefix, Subnet ID	Identifies <i>which network</i> — assigned by admin/ISP.
Interface ID	Last 64	Host ID	Identifies <i>which specific device</i> on that network.

The Interface ID can be set two ways:

Method	Description
Manual	Admin types the full 64-bit host part directly, e.g. <code>ipv6 address 3A:2B1C::1/64</code>
EUI-64	Auto-generated from the interface's 48-bit MAC address (see Section 8)

Same address, three equivalent forms:

Form	Address
Full	<code>003A:2B1C:0000:0000:0009:0101:0000:007C /64</code>
Leading zeros dropped	<code>3A:2B1C:0:0:9:101:0:7C /64</code>
Zero-run compressed	<code>3A:2B1C::9:101:0:7C /64</code>

⚠ :: can appear **only once per address** — using it twice makes it impossible to tell how many zero groups each one represents.

7.4 Configuring IPv6 (Cisco IOS)

```
Step 1 – Enable IPv6 routing globally:
Router(config)# ipv6 unicast-routing

Step 2 – Enter the interface:
Router(config)# interface g0/0

Step 3a – Auto-generate the host part via EUI-64:
Router(config-if)# ipv6 address 3A:2B1C::/64 eui-64

Step 3b – OR manually assign a complete address:
Router(config-if)# ipv6 address 3A:2B1C::1/64
```

Command	Purpose
<code>ipv6 unicast-routing</code>	Enables IPv6 routing on the device
<code>interface g0/0</code>	Enters configuration mode for that interface
<code>ipv6 address <prefix>/64 eui-64</code>	Auto-generates the Interface ID from the MAC address
<code>ipv6 address <full-address>/64</code>	Manually assigns a complete IPv6 address

8. EUI-64 (Extended Unique Identifier)

EUI-64 automatically builds a 64-bit Interface ID (Section 7.3) out of a device's 48-bit MAC address, so no one has to type a host address by hand.

Algorithm:

1. Split the 48-bit MAC address into two 24-bit halves.
2. Insert FFFE in the middle → 48 bits become 64 bits.
3. Flip the 7th bit of the first byte (the universal/local bit).

Worked example — MAC = `00:1A:2B:3C:4D:5E`

Step	Value
Original MAC	<code>00:1A:2B:3C:4D:5E</code>
Split into halves	<code>00:1A:2B 3C:4D:5E</code>
Insert <code>FFFE</code>	<code>00:1A:2B:FF:FE:3C:4D:5E</code>
Flip 7th bit (<code>00</code> → <code>02</code>)	<code>02:1A:2B:FF:FE:3C:4D:5E</code>

Full address example — prefix `3A:2B1C::/64` + this Interface ID:

`3A:2B1C:02:1A:2B:FF:FE:3C:4D:5E`

This is exactly what the `eui-64` keyword does in `ipv6 address ... eui-64` (Section 7.4).

9. IPv6 Address Scopes: Link-Local, ULA, and Global Unicast

Every IPv6 device typically carries **up to three addresses at once**, each meant for a different scope of communication:

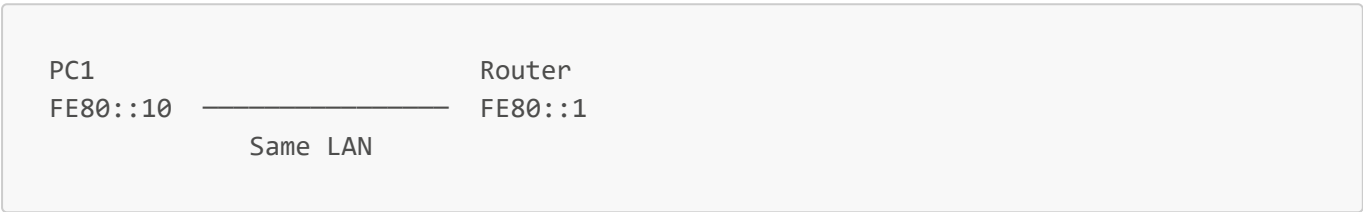
PC1
└─ Link-local: `FE80::10` → same LAN only

- ULA: FD12::10 → private organization-wide
- Global (GUA): 2001:db8:1::10 → the whole Internet

9.1 Link-Local Address — FE80::/10

Used for communication **within the same local link only**. It is created automatically by every IPv6 interface and is essential for:

- Talking to the local router
- Neighbor Discovery (NDP — Section 11)
- Router Advertisements (RA) and SLAAC



A link-local address **cannot be routed** past the local router — it never leaves the segment it was created on.

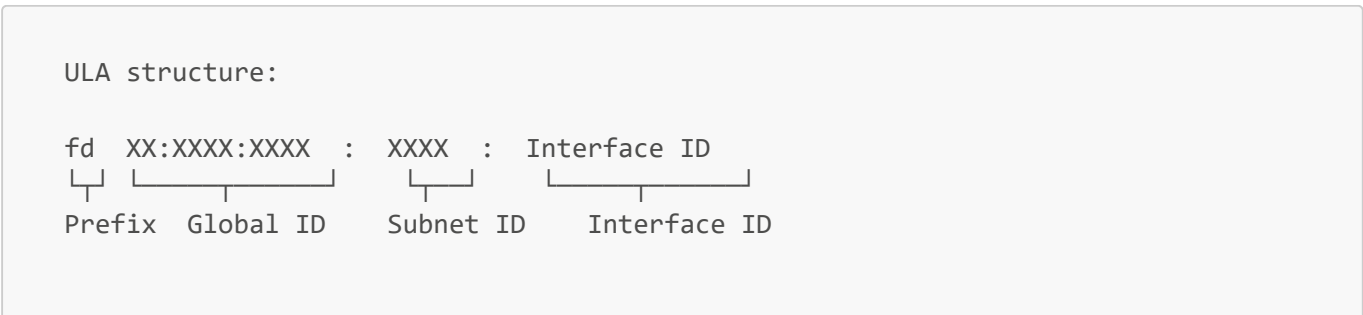
9.2 Site-Local Address — FEC0::/10 (Deprecated)

An older private-addressing scheme, similar in intent to IPv4's private ranges, but it was **deprecated** because it caused routing ambiguity when organizations merged their networks. **It should not be used in new designs.**

9.3 Unique Local Address (ULA) — FC00::/7, Commonly Seen as FD00::/8

ULA is what **replaced** Site-Local as IPv6's version of a "private" address. Example: FD12:3456:789A:1::10.

	IPv4 Private	IPv6 ULA
Range	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	fc00::/7 (usually fd00::/8)
Routable on the public Internet?	✗ No	✗ No
Routable within an org/site?	✓ Yes	✓ Yes
Globally unique?	No — ranges can clash if networks merge	✓ Yes — a randomly generated Global ID makes collisions very unlikely even after a merger



(7-8 bits) (40 bits, random) (16 bits) (64 bits)

9.4 Global Unicast Address (GUA) — 2000::/3

This is IPv6's equivalent of a public IPv4 address — used for direct, end-to-end communication with the Internet.

9.5 Summary Table

Address Type	Range	Scope	Routeable?
Link-Local	FE80::/10	Same link/LAN only	✗ Not by routers
Site-Local (<i>deprecated</i>)	FEC0::/10	Old internal site	✗ Deprecated, don't use
ULA	FC00::/7	Private organization/network	✓ Within private networks only
Global Unicast (GUA)	2000::/3	Internet	✓ Yes

One-liners:

- Every IPv6 interface has a link-local address, even with no global address configured.
- Site-Local is dead — use ULA for private addressing instead.
- Neither Link-Local nor ULA is reachable from outside its own scope.

9.6 Full IPv6 Special Address Range Reference

Beyond the scopes above, a handful of other reserved ranges show up regularly in exams and configs:

IPv6 Address / Range	Name	Purpose
::/128	Unspecified	Means no address — used as a placeholder when an address isn't yet known (e.g. a host's source address before DHCPv6/SLAAC completes).
::1/128	Loopback	The device talking to itself — IPv6's equivalent of IPv4 127.0.0.1.
FE80::/10	Link-Local	Communication on the same LAN/link only (Section 9.1).
FC00::/7	Unique Local Address (ULA)	Private/internal IPv6 networks, not Internet-routable (Section 9.3).
FF00::/8	Multicast	One-to-many delivery — replaces IPv4 broadcast (Section 7.1).
2000::/3	Global Unicast	Public IPv6 addresses used on the Internet (Section 9.4).
2001:db8::/32	Documentation	Reserved specifically for use in examples, docs, and textbooks — never assigned to real devices, so it's safe to use in configs shown here.

IPv6 Address / Range	Name	Purpose
<code>::ffff:0:0/96</code>	IPv4-mapped	Represents an IPv4 address inside an IPv6 packet/application, for dual-stack software that needs to handle both address families uniformly.
<code>100::/64</code>	Discard-Only	A "black hole" range — packets sent here are deliberately dropped, useful for testing or filtering.
<code>FE00::/9</code>	Reserved	Unassigned space held in reserve (part of what used to include the deprecated Site-Local range, Section 9.2).

`FE00::/9` is a separate reserved block from `FE80::/10` and `FEC0::/10` (old Site-Local) — despite the similar-looking prefixes, the bit math places them in different, non-overlapping ranges. Don't assume nesting just because the hex looks similar.

9.7 Can a ULA-Only Device Reach the Internet?

No — not directly. Since ULA (`fc00::/7`) isn't routable on the public Internet, packets sourced from a ULA address get dropped as soon as they try to leave the private network. To actually reach the Internet, one of these is needed:

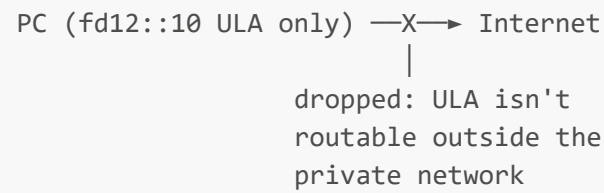
Option	How It Works	NAT Needed?
Dual addressing (<i>standard, recommended</i>)	Every device gets both a ULA (for internal traffic) and a GUA (for Internet traffic), typically via SLAAC or DHCPv6. Internet-bound packets simply use the GUA.	✗ No
NAT66 / NPTv6 (<i>uncommon</i>)	The edge router translates the ULA source prefix into a GUA prefix — conceptually similar to IPv4 NAT. Officially discouraged and rarely deployed in IPv6 networks.	✓ Yes

Standard design — dual addressing, no NAT:

```
PC — fd12:3456:789a:1::10  (ULA — internal LAN traffic)
    └─ 2001:db8:1::10      (GUA — traffic to the Internet)
```

Both addresses live on the SAME interface — no translation needed.

ULA-only LAN (no GUA assigned) – cannot reach the Internet:



This is why IPv6 is often described as "**NAT-free**" — the normal design gives every Internet-facing device its own GUA, so no translation is ever needed (contrast with IPv4 in Section 10). NAT66/NPTv6 exists only as a fallback for networks that insist on staying ULA-only.

Edge case: it's technically possible for only *one* device on a LAN to hold a GUA, with everyone else staying ULA-only. In that setup, the ULA-only devices must funnel their Internet traffic through that one device acting as a gateway/proxy — conceptually similar to IPv4-style NAT/PAT. This is **not** the recommended design; IPv6's huge address space makes giving every device its own GUA cheap and simple, so that's the standard approach.

10. IPv6 Enhancements Over IPv4

Function	IPv4 Approach	IPv6 Approach
Subnetting	Manual subnetting required to divide networks	Vast 128-bit space mostly avoids complex subnetting; a simple hierarchical /64 prefix is the norm
Address Resolution	ARP — maps IP→MAC using broadcast	NDP — resolves addresses, detects duplicates, and discovers routers, using multicast instead of broadcast
Address Assignment	DHCP — server assigns IPs	SLAAC — device self-configures from the advertised network prefix; DHCPv6 available for stateful cases
Network Booting	BOOTP — bootstrap + IP assignment	Largely unnecessary — SLAAC (and DHCPv6 where needed) covers this

In short: IPv6 folds ARP, DHCP, and BOOTP's responsibilities into more efficient, integrated mechanisms (NDP, SLAAC), and its enormous address space removes most of the pressure to subnet carefully.

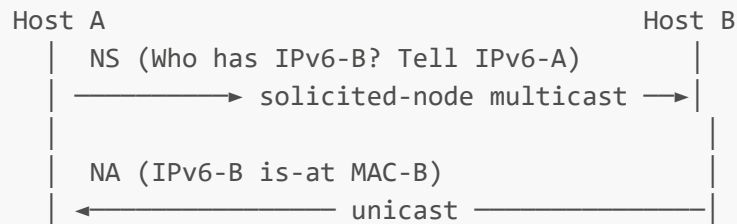
11. NDP — Neighbor Discovery Protocol (IPv6)

NDP replaces ARP (and adds more capabilities), using **ICMPv6 multicast** instead of broadcast.

Message	Direction	Purpose
RS (Router Solicitation)	Host → ff02::2 (all-routers)	"Is there a router here?"
RA (Router Advertisement)	Router → ff02::1 (all-nodes)	"I'm here — here's the network prefix and gateway."

Message	Direction	Purpose
NS (Neighbor Solicitation)	Host → solicited-node multicast	Resolves an IPv6 address to a MAC address; also used for Duplicate Address Detection (DAD).
NA (Neighbor Advertisement)	Host/Router → unicast reply	Answers an NS with the requested MAC address.

Address Resolution Flow (NDP):



DAD (Duplicate Address Detection): before a host starts using a new IPv6 address, it sends an **NS** targeting its own tentative address. If it gets an **NA** back, that address is already in use elsewhere — it must not be used.

12. IPv4 vs IPv6 — Side-by-Side Comparison

Feature	IPv4	IPv6
Size	32-bit	128-bit
Format	192.168.1.10	2001:db8::1
Total addresses	~4.3 billion	$\sim 3.4 \times 10^{38}$
Auto-config	DHCP only	SLAAC or DHCPv6
Address resolution	ARP (broadcast)	NDP (multicast)
Loopback	127.0.0.1	::1
Private / local addressing	RFC 1918 ranges	Unique Local Address, fc00::/7
Link-local	169.254.x.x (APIPA)	fe80::/10 (mandatory on every interface)
NAT needed?	✓ Usually yes	✗ Rarely — only if a network stays ULA-only with no GUA (Section 9.6)

IPv6 adoption challenges: legacy hardware/software that doesn't support it, the added complexity of running dual stack, routing-table growth, a skills gap among network staff, and migration cost.

Transition mechanisms:

- **Dual Stack** — a device runs IPv4 and IPv6 side by side.
 - **Tunneling** — IPv6 packets are wrapped inside IPv4 packets (e.g. 6to4, Teredo) to cross IPv4-only infrastructure.
 - **Translation** — a device converts between IPv4 and IPv6 directly (e.g. NAT64) for interoperability.
- Administrative Distance (AD)** — A number that tells a router how much it **trusts a route source**. Lower AD = more trusted. When two sources advertise the same destination, the router picks the one with the lower AD. Range: 0 (most trusted) to 255 (never used).
-

1) What is Routing?

Routing is the process of choosing the best path for data packets to travel from a source device to a destination device across one or more networks.

In simple words, routing decides:

- **Where a packet should go next**
- **Which path is best**
- **How to get data across different networks efficiently**

Routing is very important in packet-switched networks such as the **Internet**, where data is broken into packets and sent independently.

2) Router

A **router** is a Layer 3 device that connects different networks and forwards packets based on the **destination IP address**.

Main functions of a router

- Receives packets from one network
- Reads the destination IP address
- Checks the routing table
- Chooses the best next hop
- Forwards the packet toward the destination

OSI Layer

- **Layer 3: Network Layer**
-

3) How Routing Works

Routing works hop by hop. A packet does not usually go directly from sender to destination in one step. Instead, it passes through routers one by one.

Working process

1. A sender creates data and adds the **destination IP address** in the packet header.
2. The packet reaches the nearest router.

3. The router checks its **routing table**.
4. The router forwards the packet to the next router or next hop.
5. This process continues until the packet reaches the destination.

Important point

- Each router along the path is called a **hop**.
 - If a packet crosses too many hops, it may be dropped.
 - To prevent packets from looping forever, IP packets have a **TTL (Time To Live)** value.
 - TTL decreases by 1 at each hop.
 - When TTL becomes 0, the packet is discarded.
-

4) Working Principle of Routing

Routing is based on finding the **best path**, not just any path.

Step-by-step flow

1. **Communication starts** A device sends data to another device, often using an application protocol like HTTP, HTTPS, or SSH.
 2. **Data is split into packets** Large data is divided into smaller packets for easier transmission.
 3. **Destination IP is added** Each packet contains the destination IP address in its header.
 4. **Router checks routing table** The router compares the destination IP with available routes.
 5. **Best path is selected** The router uses metrics like hop count, bandwidth, delay, or cost.
 6. **Packets move through hops** Packets travel through multiple routers until they reach the destination.
 7. **Reassembly at destination** The destination device collects packets and rebuilds the original data.
-

5) Routing Table

A **routing table** is a database stored in a router that contains information about possible paths to different networks.

A routing table usually contains:

- Destination network
- Next hop
- Metric / cost
- Route source
- Interface to use

Why it matters

The router uses the routing table to decide where to send a packet next.

6) Types of Routing

Routing is mainly of **three types**.

Type	Meaning	What Router Learns / Uses	Best For	Advantages	Disadvantages
Static Routing	Manually configured routes	Nothing learned automatically; routes are manually entered (destination + next hop)	Small, simple networks	Simple, predictable, no protocol overhead	Not scalable, no automatic failover
Dynamic Routing	Routes learned automatically using protocols	Destination networks, next-hop addresses, and metrics via RIP, OSPF, BGP	Large and changing networks	Scalable, adapts to failures automatically	Complex, uses CPU, memory, and bandwidth
Default Routing	Uses a predefined route when no other route matches	Uses a single default path (default gateway) when no other route is available	Networks with a single exit path	Easy to configure, reduces routing table	Can cause inefficient routing in complex networks

6.1 Static Routing

Static routing means the network administrator manually enters the route into the router.

In static routing, routing entries are added manually in the routing table and typically include:

- Destination IP address
- Next-hop address (or exit interface)

Features

- Routes are configured by hand
- Does not change automatically
- Very simple and predictable
- Uses no routing protocol

Advantages

- Full control over the path
- More secure in small networks
- Low CPU and bandwidth usage

Disadvantages

- Hard to manage in large networks
 - No automatic failover
 - Not suitable when links change often
-

6.2 Dynamic Routing

Dynamic routing means routers automatically learn routes and update them when the network changes.

Features

- Routes are discovered automatically
- Routers exchange information
- Adapts to failures and new paths

Advantages

- Easier to manage in large networks
- Automatically reacts to network changes
- Scales better than static routing

Disadvantages

- Uses more CPU, memory, and bandwidth
- More complex than static routing
- Routing loops and convergence delays may occur

Issue	Meaning	Why It Happens in Dynamic Routing	Impact on Network	How It Is Reduced
Routing Loops	Packets circulate repeatedly between routers without reaching destination	Temporary inconsistent routing information between routers	Wastes bandwidth, increases delay, packet loss	Split horizon, route poisoning, hold-down timers
Convergence Delays	Time taken for all routers to update and agree on new network topology	Routers need time to exchange updates and recalculate routes	Temporary routing errors, packet drops	Faster protocols like OSPF/EIGRP, efficient updates

6.3 Default Routing

Default routing is used when the router does not know a more specific route.

Default route

- **0.0.0.0/0** in IPv4
- Means "send packets to this gateway if no other route matches"

Used when

- A network has only one way out
- A small branch office sends all unknown traffic to the main router or ISP

Example

If a router does not have a route for a destination, it sends the packet to the **default gateway**.

7) Administrative Distance

When a router learns the same destination from different routing sources, it uses **administrative distance** to choose the more trusted source. Lower AD means higher trust.

Administrative Distance (AD) Table

AD = Administrative Distance

Lower AD = More Trusted Route

Route Source	AD (Administrative Distance)	Meaning
Connected (Directly Connected)	0	Networks directly attached to router interfaces
Static Route	1	Manually configured routes
eBGP	20	Routes learned via External BGP
EIGRP Internal	90	Routes learned via EIGRP within same AS
IGRP	100	Routes learned via IGRP protocol
OSPF	110	Routes learned via OSPF protocol
IS-IS	115	Routes learned via IS-IS protocol
RIP	120	Routes learned via RIP protocol
EIGRP External	170	Routes redistributed into EIGRP
iBGP	200	Routes learned via Internal BGP
Unknown	255	Untrusted / unusable route (ignored)

Notes

- **Administrative Distance (AD)** indicates the **trustworthiness of a route source**.
- **Lower AD = Higher priority (more trusted)**.
- A route with **AD 255** is **never used**.

Quick Memory Order (Low → High AD)

Connected → Static → eBGP → EIGRP → IGRP → OSPF → IS-IS → RIP → EIGRP External → iBGP → Unknown

0 → 1 → 20 → 90 → 100 → 110 → 115 → 120 → 170 → 200 → 255

Note - Default Routing != Directly connected

EIGRP internal routes are preferred over OSPF and RIP because their default administrative distance is lower.

8) Routing Protocol Categories

Routing protocols are rules used by routers to learn networks and choose the best path for packets. They are usually grouped into **distance vector**, **link-state**, and **hybrid** protocols. These are mainly **intradomain routing** protocols, meaning they are used inside an autonomous system rather than between different organizations or ISPs.

Category	Main idea	Example protocols	Typical use
Distance vector	Learns routes from neighbors only	RIP, IGRP	Small or simple networks
Link-state	Builds a full topology map	OSPF, IS-IS	Medium to large networks
Hybrid	Combines features of both	EIGRP	Enterprise and mixed-size networks

9) Metrics and Path Choice

A routing protocol decides the best path using a **metric**, which is its internal path cost. Different protocols use different metrics, so "best" can mean different things depending on the protocol.

Common metrics

- **Hop count**: number of routers crossed; RIP uses this.
- **Cost**: used by OSPF, usually based on bandwidth.
- **Bandwidth**: higher bandwidth links are preferred in OSPF cost calculation.
- **Delay**: used in some composite metrics such as EIGRP.
- **Reliability and load**: also part of composite path selection in EIGRP-style designs.

OSPF cost formula

OSPF commonly calculates cost as:

$$\text{cost} = \frac{\text{reference bandwidth}}{\text{interface bandwidth}}$$

With the default reference bandwidth of 100 Mbps, a 10 Mbps interface has a cost of 10.

10) Distance Vector Routing

Distance vector routing protocols choose routes based mainly on what their **neighbors** tell them. The router does not know the whole network map; it trusts information received from directly connected routers, which is why this is often called **routing by rumor**. RIP is the classic example, and its metric is hop count, meaning the route with the fewest routers is preferred.

Features

- Routers exchange updates periodically, often with the full routing table in RIP.
- Updates are shared only with neighboring routers, not flooded to the whole network.
- The main metric is usually **hop count**.
- RIP has a maximum hop count of **15**; a destination at 16 hops is unreachable.

Advantages

- Easy to configure and understand.
- Uses less CPU and memory than link-state protocols.
- Works well in small networks.

Disadvantages

- Converges slowly after a network change.
- More likely to suffer from routing loops and the **count-to-infinity** problem.
- Not ideal for large or fast-changing networks.

Distance vector vs link-state

Feature	Distance Vector	Link-State
Network view	Neighbor-based	Full topology-based
Updates	Periodic	Triggered + incremental
Algorithm	Bellman-Ford	Dijkstra SPF
Convergence	Slower	Faster
Loop issues	More common	Much less common
Example	RIP	OSPF, IS-IS

RIP overview

The **Routing Information Protocol (RIP)** is a dynamic routing protocol used by routers to find paths to destination networks. It operates at the **Network Layer (Layer 3)** and belongs to the **distance-vector** family. RIP is designed mainly for small to medium networks because it uses a very simple metric and has limited scalability.

Core idea

- Routers share route information with neighbors only.
- Each route is measured by **hop count**.

- The route with the lowest hop count is preferred.

Hop count

Hop count is the number of routers a packet must cross to reach a destination. In RIP, every router crossed adds **1 hop**, so the path with fewer routers is considered better. RIP allows a maximum hop count of **15**; a hop count of **16** means the destination is unreachable.

Why this matters:

- It prevents routes from circling forever in loops.
- It also limits RIP's usable network size.
- A path with better bandwidth is still ignored if it has more hops, because RIP does **not** consider bandwidth or delay.

How RIP works

RIP uses the **distance-vector** method, which means each router knows only what its neighbors tell it. Every **30 seconds**, routers send their routing table updates to neighboring routers, and the information spreads gradually until the network reaches **convergence**.

Operation flow

1. The router starts with directly connected networks.
2. It exchanges updates with neighbors every 30 seconds.
3. If a router learns a shorter path, it updates its table.
4. If updates stop arriving, the route becomes invalid after **180 seconds**.
5. The route is removed after **240 seconds** if it is not refreshed.

RIP features

- Periodic updates every 30 seconds.
- Shares the **full routing table** in updates.
- Uses **routing by rumor**, meaning routers trust information from neighbors.
- Uses loop-reduction methods like **split horizon** and **route poisoning**.
- Works with a simple hop-count metric.

RIP versions

Version	IP type	Update method	Subnet mask support	Authentication
RIPv1	IPv4	Broadcast to 255.255.255.255	No, classful only	No
RIPv2	IPv4	Multicast to 224.0.0.9	Yes, classless	Yes
RIPng	IPv6	Multicast to FF02::9	Yes, classless	Supports IPv6 security mechanisms; not the same as classic RIPv2 auth

RIP timers

RIP uses timers to control route freshness and stability. The default values are:

- **Update timer:** 30 seconds.
- **Invalid timer:** 180 seconds.
- **Hold-down timer:** 180 seconds.
- **Flush timer:** 240 seconds.

Timer meaning

- **Update timer:** when the next routing update is sent.
- **Invalid timer:** when a route is marked unreachable.
- **Hold-down timer:** prevents unstable routing changes from being accepted too quickly.
- **Flush timer:** removes the route from the table completely.

RIP advantages

- Very easy to configure.
- Low CPU and memory usage.
- Simple to understand and troubleshoot.
- Works well in small networks.
- Can support basic equal-cost load balancing in some cases.

RIP disadvantages

- Limited to **15 hops**, so it does not scale well.
- Slow convergence after failures.
- Periodic full-table updates waste bandwidth.
- Weak for modern enterprise needs like large topologies and advanced path selection.

Security concerns

RIP is weak in security because updates are easy to intercept and manipulate if authentication is not used. Problems include route spoofing, eavesdropping, and false updates that can trigger instability or denial of service.

Why RIP is considered old

RIP is often called "dead" in modern networks because it:

- uses only hop count, not bandwidth-aware metrics,
- sends updates frequently,
- supports only 15 hops,
- converges slowly,
- is less efficient than OSPF or EIGRP for real-world networks.

11) Link-State Routing

Link-state routing protocols give each router a much more complete view of the network. Routers share information about their links, form a topology database, and then run **Dijkstra's SPF algorithm** to calculate

the best path. OSPF and IS-IS are the common examples.

Features

- Routers use **hello packets** to discover and maintain neighbors.
- Topology changes are sent as **triggered updates** rather than fixed periodic full-table advertisements.
- Each router keeps a copy of the network topology in a database.
- OSPF stores and uses **LSAs** (Link-State Advertisements) to build the topology view.

Tables used in link-state

Table	Purpose
Neighbor table	Stores directly adjacent routers and adjacency status
Topology table / LSDB	Stores the full network map and all learned link information
Routing table	Stores the best routes selected from the SPF calculation

Advantages

- Fast convergence after changes.
- More accurate because routers know the overall topology.
- Scales better than distance vector in larger networks.
- Avoids persistent routing loops more effectively.

Disadvantages

- Uses more memory and CPU because of topology calculations.
- Configuration and troubleshooting are more complex.
- Flooding link-state information can use more bandwidth.

OSPF overview

OSPF (Open Shortest Path First) is a **link-state routing protocol** used inside an autonomous system. Each router independently runs OSPF, learns the topology, and calculates the best routes based on **cost**, not hop count.

What OSPF does

- Discovers neighbors with Hello packets.
- Builds a link-state database (LSDB).
- Runs SPF (Dijkstra) to select shortest paths.
- Uses areas to reduce flooding and improve scalability.

Key OSPF terms

Term	Meaning
Router ID	A unique 32-bit number used to identify an OSPF router.

Term	Meaning
Hello packet	A packet used to find neighbors and keep the OSPF relationship alive.
Area ID	A number that identifies which OSPF area an interface belongs to. Routers in the same area exchange OSPF information with each other.
DR	Designated Router. It reduces the number of OSPF adjacencies on multi-access networks.
BDR	Backup Designated Router. It takes over if the DR fails.
LSA	Link-State Advertisement. This is the message OSPF uses to share network topology details.
LSDB	Link-State Database. This is the table where a router stores all received LSAs.

Router ID

The Router ID is like the **name tag** of an OSPF router. It must be unique inside the OSPF network.

How it is chosen

1. If the router-id is manually configured, OSPF uses that first.
2. If not, OSPF uses the highest loopback IP address.
3. If there is no loopback, it uses the highest active interface IP address.

Why it matters

- It identifies the router in OSPF.
- It is used in DR/BDR election when priorities are equal.
- If no valid Router ID exists, OSPF may not start properly.

Hello packet

Hello packets are OSPF's way of saying, "Are you there?" They are sent regularly to discover neighbors and maintain the relationship between routers.

Main job of Hello packets

- Discover OSPF neighbors.
- Keep the neighbor relationship alive.
- Help in DR/BDR election on broadcast and multi-access networks.

Important idea

If a router stops receiving Hello packets for the dead interval, it assumes the neighbor is down.

OSPF broadcast and multicast addresses

OSPF does not use broadcast to send updates. Instead, it uses **multicast** so only OSPF-enabled routers receive the packets, not every device on the network.

Address	Who receives it	Used for
224.0.0.5	All OSPF routers	General Hello packets and LSU flooding — every OSPF router listens on this address
224.0.0.6	DR and BDR only	DROTHERs send updates to this address so only the DR and BDR process them

How it works on a LAN segment

- All OSPF routers send Hello packets to **224.0.0.5** to discover neighbors.
- When a DROTHER needs to share a route update, it sends it to **224.0.0.6** (DR/BDR only).
- The DR then re-floods the update to **224.0.0.5** so all routers on the segment receive it.

This two-step process reduces unnecessary processing on routers that are not the DR or BDR.

Why not broadcast?

- Broadcast goes to **every device** on the segment, including non-routers like PCs and switches.
- Multicast reaches only devices that have joined that multicast group — so only OSPF routers are involved.

Area ID

The Area ID tells OSPF which **group** or **section** of the network an interface belongs to. Routers with the same area ID can exchange detailed OSPF information directly.

Why areas are used

- They divide a large network into smaller parts.
- They reduce flooding and make OSPF easier to scale.
- They keep LSDB size smaller inside each area.

Important note

- Area 0 is the backbone area.
- In general, other areas must connect to area 0 somehow.

DR and BDR in OSPF

On **broadcast** and **multi-access** networks (like Ethernet segments), OSPF elects two special routers to reduce traffic:

- **DR (Designated Router)** → the main router for distributing routing information on that segment.
- **BDR (Backup Designated Router)** → the backup; it becomes DR if the original DR fails.

Instead of every router forming full adjacency with every other router, all other routers (called **DROTHER**) form adjacencies mainly with the DR and BDR. This reduces the number of adjacencies and LSA flooding, which saves bandwidth and CPU.

How DR/BDR election works

OSPF uses two main things to decide which router becomes DR and which becomes BDR:

1. **OSPF priority (per interface)**
2. **Router ID (RID)**

Election rules

1. Highest priority wins as DR

- Each router has an **OSPF priority** on the interface, from 0 to 255.
- The router with the **highest priority** becomes **DR**.
- The router with the **second-highest priority** becomes **BDR**.

2. If priorities are equal, highest Router ID wins

- If two routers have the **same priority**, the router with the **highest Router ID** wins the DR role.
- The router with the next-highest Router ID becomes BDR.

3. Priority 0 → not eligible

- A router with **priority 0** cannot become DR or BDR.
- It will only form adjacencies as a **DROTHER**.

4. Default priority = 1

- If nobody configures priority, every router has **priority 1** by default.
- In that case, DR/BDR is chosen purely by **Router ID**.

OSPF network types

OSPF behaves differently depending on the **network type** of an interface. The network type controls two things: whether a **DR/BDR is elected**, and how **neighbors are discovered**.

Network Type	DR/BDR Elected?	Neighbor Discovery	Typical Use
Broadcast	Yes	Automatic (multicast)	Ethernet (LAN)
Point-to-Point	No	Automatic (multicast)	Serial links, PPP, HDLC
NBMA	Yes	Manual (unicast)	Frame Relay, ATM
Point-to-Multipoint	No	Automatic (multicast)	Frame Relay hub-and-spoke

Broadcast

- The most common type in modern networks — used on **Ethernet**.
- OSPF automatically discovers neighbors using **multicast** (224.0.0.5 / 224.0.0.6).
- A **DR and BDR are elected** to reduce the number of adjacencies on the segment.
- All other routers on the segment are called **DROTHERs** and only form full adjacency with the DR and BDR.

Point-to-Point

- Used on links with only **two routers** (e.g., serial links, PPP, HDLC WAN connections).
- **No DR/BDR election** — not needed because there are only two endpoints on the link.
- The two routers form a **direct full adjacency** with each other automatically.
- Simple and fast — this is the preferred type for serial WAN links.

NBMA (Non-Broadcast Multi-Access)

- Used on older WAN technologies like **Frame Relay or ATM**, where multiple routers share the same network but the medium **does not support native broadcast**.
- **DR/BDR is still elected** (same logic as broadcast), but neighbors must be **manually configured** because multicast does not work natively here.
- Requires **neighbor** statements under the OSPF process to tell OSPF where to send updates.

Point-to-Multipoint

- Also common on **hub-and-spoke WAN** designs (Frame Relay).
- **No DR/BDR election** — OSPF treats each connection to a spoke as a separate point-to-point link.
- Neighbors are discovered **automatically** where multicast is supported, or can be configured manually.
- Simpler to set up than NBMA because it avoids the DR/BDR complexity.

Quick rule to remember

- **DR/BDR elected** → Broadcast, NBMA
 - **No DR/BDR** → Point-to-Point, Point-to-Multipoint
-

LSA

LSA stands for **Link-State Advertisement**. It is the message OSPF uses to share information about links, routers, and network changes.

What an LSA contains

- Router information.
- Link/interface information.
- Cost and connectivity details.
- Topology changes.

Why it matters

- LSAs are how OSPF learns the full network topology.
- When something changes, new LSAs are flooded so all routers can update their view.

LSDB

LSDB means **Link-State Database**. It is the collection of all LSAs a router knows about.

Simple meaning

Think of the LSDB as OSPF's **network map**. Every router in the same area keeps a similar map so they can calculate the best route.

Why it is important

- OSPF uses the LSDB as input for SPF calculation.
- A correct LSDB is needed for correct route calculation.
- If LSDBs do not match, routing problems can happen.

OSPF neighbor states

When two routers start talking OSPF, they do not become fully connected immediately. They move through a set of states step by step until they finally reach **Full**, which means their databases are synchronized.

State	Meaning
Down	No Hello packet has been received yet. OSPF has not started forming a relationship.
Init	A Hello packet was received, but the router has not yet seen its own Router ID in the neighbor's Hello packet.
Two-Way	Both routers can see each other in Hello packets, so bidirectional communication is confirmed.
ExStart	The routers decide who will be master and who will be slave for database exchange.
Exchange	Routers exchange DBD packets, which contain summaries of their LSDB contents.
Loading	Routers request missing LSAs using LSR packets and receive them using LSU packets.
Full	The LSDBs are synchronized. At this point, the OSPF adjacency is complete.

What the states really mean

You can think of the OSPF process like two routers getting to know each other.

- **Down** means they have not started talking yet.
- **Init** means one router has heard the other, but the connection is not confirmed both ways.
- **Two-Way** means both routers recognize each other.
- **ExStart** and **Exchange** are about preparing and sharing database summaries.
- **Loading** is when missing information is requested.
- **Full** means both routers now have the same view of the network.

Neighbor vs adjacency

A **neighbor** is just a router that has been discovered by Hello packets. It means the routers know each other exists.

An **adjacency** is a stronger relationship. It means the routers have gone beyond discovery and have fully exchanged and synchronized their LSDBs.

Simple example

- If Router A and Router B send Hello packets to each other, they are **neighbors**.
- If they also exchange database information and finish synchronizing, they are **adjacent**.

Why not every neighbor becomes adjacent

On broadcast networks like Ethernet, OSPF uses the **DR/BDR** mechanism to reduce overhead. That means routers do not always form full adjacencies with every other router on the same network segment. Instead, many routers only fully adjacence with the DR and BDR, which keeps the number of relationships lower and makes the network more efficient.

Why adjacency can fail

OSPF may stop before Full if some values do not match. Common problems include:

- Area ID mismatch.
- Hello/dead timer mismatch.
- Authentication mismatch.
- MTU mismatch.
- Wrong network type in some designs.

Easy memory trick

- **Neighbor** = "I know you."
- **Adjacency** = "I know you, and I share my routing database with you."
- **Full** = "We are completely synchronized."

OSPF message types

OSPF uses **5 types of packets** to discover neighbors, synchronize databases, and update topology.

1. Hello

- **Purpose:** Discover and maintain OSPF neighbors.
- **What it does:**
 - Says "I am here" (neighbor discovery).
 - Keeps the neighbor relationship alive (like a heartbeat).
 - Helps in **DR/BDR election** on broadcast networks.
- **Used in:** Down → Init → Two-Way states.

2. DBD (Database Description)

- **Purpose:** Exchange a **summary** of the LSDB (not the full routes).
- **What it does:**
 - Each router shows what LSAs it knows (like a table of contents).
 - Routers compare DBD packets to see **which LSAs are missing**.
 - Used in **ExStart** and **Exchange** states.

3. LSR (Link-State Request)

- **Purpose:** Ask for **specific missing LSAs**.
- **What it does:**
 - After comparing DBDs, a router says, "You have something I don't; send me those LSAs."
 - Used in the **Loading** state.

4. LSU (Link-State Update)

- **Purpose:** Send the **actual LSAs** that were requested.
- **What it does:**
 - Contains full **Link-State Advertisements** (routing info).
 - Can be used to answer an LSR or to flood new topology changes.
 - Sent in **Loading** state for sync, and anytime an LSA is updated (flooding).

5. LSAck (Link-State Acknowledgment)

- **Purpose:** Confirm that LSU packets were received.
- **What it does:**
 - Each router sends an LSAck to say "Yes, I got your LSU and its LSAs."
 - This makes OSPF **reliable** (if no LSAck, the LSU is re-sent).

OSPF cost

OSPF uses **cost** as its only metric to decide which path is the best. Lower cost is always preferred, and by default, cost is based on **interface bandwidth**.

What "cost" means

- **Cost = how "expensive" a link is to use.**
- Low-bandwidth links have **high cost** (not preferred).
- High-bandwidth links have **low cost** (preferred).

For example, OSPF should prefer a 100 Mbps link over a 10 Mbps link for the same path.

The cost formula

$$\text{Cost} = \frac{\text{Reference Bandwidth}}{\text{Interface Bandwidth}}$$

- **Reference Bandwidth** is a fixed value set by the vendor (by default **100 Mbps** on Cisco).
- **Interface Bandwidth** is the actual speed of the link (e.g., 10 Mbps, 100 Mbps, 1 Gbps).

Example values (Cisco default)

Link type	Bandwidth	Formula (100 Mbps / BW)	Cost
10 Mbps Ethernet	10 Mbps	100 / 10	10
100 Mbps FastEthernet	100 Mbps	100 / 100	1

Link type	Bandwidth	Formula (100 Mbps / BW)	Cost
1 Gbps GigabitEthernet	1 Gbps	100 / 1000	1 (since cost is rounded to integer, any value < 1 becomes 1)

How OSPF uses cost

- Each router assigns a **cost to every outgoing OSPF interface** using this formula.
- When choosing a path to a destination, OSPF **adds up all the link costs** along the path.
- The path with the **lowest total cost** becomes the best route.

Simple example

Suppose Router A has two paths to a server:

- Path 1: A → B → Server, with link costs 10 + 10 = **20**
- Path 2: A → C → Server, with link costs 1 + 1 = **2**

Even if both paths have the same number of hops, OSPF chooses **Path 2** because its **total cost (2)** is lower than cost 20.

LSDB and SPF

Each router stores LSAs in the **LSDB**, and that database becomes the input to the SPF algorithm. SPF does not directly read packets to build the routing table; instead, it calculates the best route tree from the LSDB from the local router's perspective.

Why OSPF areas matter

Areas exist mainly to **reduce load** and **improve convergence**:

- **Limit LSA flooding** to just the area.
- **Reduce RAM** because LSDBs are smaller.
- **Reduce CPU** because SPF runs on smaller LSDBs.
- **Improve convergence** because changes in one area do not force full SPF runs everywhere.

OSPF packet flow (step-by-step)

This is the usual OSPF "conversation" on a segment:

1. **Hello** → Routers send Hello packets.
2. **Two-Way** → Neighborships form (bidirectional communication).
3. **DR/BDR election** → On broadcast/multi-access links, DR and BDR are elected.
4. **ExStart & Exchange** → Routers negotiate master/slave and exchange DBD packets (LSDB summaries).
5. **Loading** → Routers ask for missing LSAs using **LSR**, receive them with **LSU**, and confirm with **LSAck**.
6. **Full** → LSDBs are synchronized; adjacency is complete and SPF can run.

OSPF areas (types explained)

OSPF breaks the network into **areas**. Area 0 is the **backbone**, and everything else connects to it.

1. Backbone area (Area 0)

- The **core** of the OSPF network.
- **All other areas must connect** to Area 0 (directly or via virtual links).
- It carries routing information between non-backbone areas.
- Supports **all LSA types** (1, 2, 3, 4, 5).

2. Standard (normal) area

- A regular OSPF area that is not special.
- **Any LSA type is allowed** (1, 2, 3, 4, 5).
- No special restrictions; it behaves like the classic OSPF design.
- This is the **default** if you don't configure a special area type.

3. Stub area

- A **simplified** area that:
 - **Blocks Type 5 external LSAs** (no external routes from other AS).
 - Instead of all externals, it uses a **default route** toward the backbone.
- Still accepts:
 - Internal LSAs (1, 2)
 - Inter-area LSAs (3).
- Goal: reduce LSDB size and routing table size in edge areas.

4. Totally stubby area (Cisco-style)

- Even stricter than a stub area:
 - **Blocks Type 5 (external) and Type 3 (inter-area) LSAs.**
- The router only knows:
 - Internal routes in the area (Types 1, 2).
 - One **default route** advertised by the ABR.
- This keeps the LSDB and routing table **very small**.

5. NSSA (Not-So-Stubby Area)

- Like a **stub** but more flexible:
 - **Blocks Type 5 external LSAs from outside.**
 - **Allows an ASBR inside the NSSA** to generate **Type 7 LSAs** for external routes.
 - At the ABR, **Type 7 LSAs are translated into Type 5** so the rest of the OSPF domain can see them.
- Use case: an edge area that needs to redistribute external routes without becoming a normal area.

OSPF router types

Backbone Area (Area 0)

- Identifier = **Area 0** (backbone area).
- Acts as the **core** of the OSPF domain.
- **All other areas must connect to Area 0** (directly or via ABRs/virtual links).
- Carries routing traffic between non-backbone areas.

Backbone router

- Any router that has **at least one interface in Area 0**.
- It may also have interfaces in other areas.
- Participates in the backbone routing but does not necessarily connect multiple areas itself.

Area Border Router (ABR)

- A router with **interfaces in multiple areas** (for example, Area 0 + Area 10).
- Connects an **internal area to the backbone**.
- **Summarizes routes** between areas and controls which LSAs are sent where.
- Plays a key role in hierarchical design and route-table size control.

Internal router

- A router whose **all interfaces belong to a single area**.
- Operates only within that area's LSDB.
- Generally **simpler** (no ABR/ASBR features).

Autonomous System Boundary Router (ASBR)

- A router that **connects an OSPF AS to another routing domain** (e.g., another AS, BGP, RIP, EIGRP).
- **Redistributes** external routes into OSPF as **Type 5 LSAs** (or **Type 7 in NSSA**).
- Typically located at the edge of the OSPF network.

OSPF addressing and summarization

- OSPF is **classless** and supports **VLSM** and **CIDR**.
- It allows **route summarization** at ABRs and ASBRs.

How summarization helps

- Instead of advertising each small subnet, ABRs can advertise **a single network ID or summary prefix**.
- This:
 - Reduces the **routing-table size**.
 - Reduces **LSA traffic and LSDB size**.
 - Hides small topology changes from the rest of the domain.

OSPFv2 vs OSPFv3

Feature	OSPFv2	OSPFv3
IP version	IPv4	IPv6

Feature	OSPFv2	OSPFv3
Addressing	IP addresses	Link-local addresses for neighbor relationships
Security	Traditionally plain text or MD5	Uses IPv6 security mechanisms such as IPsec support
Scope	Area and AS scope	Adds more flexible flooding scopes

12) Hybrid Routing

Hybrid routing combines ideas from distance vector and link-state routing. EIGRP is the best-known example in Cisco environments. It learns routes from neighbors like a distance-vector protocol, but it also uses fast convergence techniques, neighbor discovery, and partial updates that make it behave more like a modern advanced protocol.

Features

- Uses **hello packets** for neighbor discovery.
- Sends **partial, triggered updates** when the topology changes.
- Uses the **DUAL algorithm** to find loop-free paths.
- Supports backup paths called **feasible successors**.

EIGRP terms

Term	Meaning
Successor	Best route to the destination, installed in the routing table
Feasible successor	Backup route that can replace the successor immediately if needed
Feasibility condition	Rule used to ensure the backup route is loop-free

Advantages

- Fast convergence.
- Efficient bandwidth use because it does not flood full tables constantly.
- Supports scalable networks.
- Can provide backup routes quickly through feasible successors.

Disadvantages

- More complex than RIP.
- Uses more CPU and memory than distance vector protocols.
- Cisco-focused in practice, so it is less universal than OSPF.

EIGRP – Core idea

EIGRP (Enhanced Interior Gateway Routing Protocol) is a **hybrid routing protocol**: it behaves mostly like an **advanced distance-vector** protocol but borrows features from link-state routing. It runs at **Layer 3 (Network Layer)** and uses **IP protocol 88**.

- Used to discover and maintain **best paths inside an AS**.
- Fast convergence, efficient bandwidth use, and unequal-cost load balancing make it popular in **enterprise networks**.

Administrative Distance in EIGRP

Administrative Distance (AD) tells the router **how much it trusts a route**. Lower AD = more trusted.

Type	AD Value
EIGRP summary routes	5
EIGRP internal routes	90
EIGRP external routes (redistributed)	170

- So, **EIGRP internal routes** are trusted more than most other IGP's (for example, OSPF = 110, RIP = 120).

Key EIGRP messages

EIGRP uses several message types, sent as **multicast (224.0.0.10)** or **unicast**, depending on purpose.

Message	Purpose
Hello	Neighbor discovery and keep-alive. Sent every 5 seconds by default. If no Hello arrives within 15 seconds (Dead/hold time) , the neighbor is declared down.
Update	After adjacency, routers send full updates (all routes). On topology changes, partial updates (only changed routes) are sent.
Query	When a route disappears and there is no backup, EIGRP queries neighbors for a new path (multicast).
Reply	Sent in response to a Query , giving alternative route info.
Ack	A Hello-like packet with no data , used to acknowledge Updates, Queries, and Replies (these are reliable messages).
NULL Update	Used internally for measuring SRTT (Smooth Round Trip Time) and RTO (retransmission timeout) .

Reliability note:

- **Hello and Ack** are **not** reliable (no separate acknowledgment).
- **Update, Query, Reply** are **reliable** and must be acknowledged.

EIGRP broadcast and multicast

EIGRP does **not** use broadcast. It uses **multicast** for most messages, and switches to **unicast** in specific situations.

Address / Type	Used for
224.0.0.10 (multicast)	Hello packets, Updates, Queries sent to all EIGRP neighbors on the segment
Unicast	Retransmissions of reliable messages if a neighbor does not acknowledge; also used on NBMA links where multicast is not supported

Why multicast instead of broadcast?

- **Broadcast** goes to every device on the network — including PCs, printers, and switches that do not run EIGRP.
- **Multicast 224.0.0.10** only reaches routers that have joined the EIGRP multicast group, which saves processing on non-EIGRP devices.

When EIGRP falls back to unicast

- If a neighbor does not **acknowledge** a reliable message (Update, Query, Reply), EIGRP retransmits it as **unicast** directly to that neighbor.
- On **NBMA links** (e.g., Frame Relay) where multicast may not work, EIGRP uses unicast with manually configured neighbor statements.

Composite metric (EIGRP metric)

EIGRP chooses the best path using a **composite metric** built from **five K-values**. By default, only **K₁ = Bandwidth** and **K₃ = Delay** are used.

K-value meanings

- **K₁ = Bandwidth**
- **K₂ = Load**
- **K₃ = Delay**
- **K₄ = Reliability**
- **K₅ = MTU**

Default K-values

$$K_1 = 1, \quad K_2 = 0, \quad K_3 = 1, \quad K_4 = 0, \quad K_5 = 0$$

So **only bandwidth and delay contribute** by default. Lower metric = better route.

Classic metric mindset

- **Bandwidth**: lower bandwidth → higher metric.
- **Delay**: higher delay → higher metric.

EIGRP's metric can be **scaled** so large bandwidths (10 Gbps, 100 Gbps) are still distinguished.

Neighbor adjacency requirements

For two routers to become **EIGRP neighbors**, these must match:

- **K-values** (all 5) on the link.
- **AS number** (same value in `router eigrp <AS>`).
- **Subnet mask / network** (same Layer-3 segment).
- **Authentication** (if enabled; EIGRP supports **MD5** only).

If any of these differ, the routers **will not form a neighbor relationship**, even if physically connected.

EIGRP timers

Timer	LAN (Ethernet) Default	WAN / Slow-link Default	Purpose
Hello Timer	5 seconds	60 seconds	Interval between Hello packets.
Dead / Hold Time	15 seconds	180 seconds	If no Hello is received for this time, the neighbor is declared dead.

Important: The 5s/15s values are the defaults on **LAN (Ethernet)** interfaces. On **WAN or slow-speed links** (T1 speed and below), EIGRP defaults to **60 seconds Hello / 180 seconds Hold**. Hello and Hold timers **must match** between neighbors — a mismatch will prevent the neighbor relationship from forming.

The LAN timers allow **fast failure detection** on high-speed links. The longer WAN timers reduce Hello overhead on slower, more sensitive connections.

Pros of EIGRP

- **Fast convergence** (via **DUAL algorithm** and feasible successors).
- **VLSM and CIDR support** for efficient IP design.
- **Partial updates** reduce bandwidth usage vs full-table distance-vector protocols like RIP.
- **Scalable to large enterprise networks**.
- **Unequal-cost load balancing** (via `variance` command).

DUAL (Diffusing Update Algorithm)

DUAL is the **core of EIGRP's loop-free convergence**. It uses:

- **Topology table** (all routes and their metrics).
- **Feasibility condition** to choose **loop-free backup routes (feasible successors)**.

Key distance terms

Before understanding DUAL, you need to know two distance values EIGRP uses:

Term	Short form	Meaning
Reported Distance	RD	The metric a neighbor reports for reaching the destination — it is the neighbor's own cost to get there. Also called Advertised Distance (AD).

Term	Short form	Meaning
Feasible Distance	FD	The total metric from the local router to the destination through a specific neighbor. It equals the link cost to the neighbor plus the neighbor's RD.

Simple way to remember:

- **RD** = what your neighbor tells you its cost is.
- **FD** = your total cost = (your link to neighbor) + (neighbor's RD).

Feasibility condition

A backup path (feasible successor) is loop-free if:

RD of the backup path < FD of the current best path

This rule guarantees the backup neighbor is closer to the destination than you are, so it cannot be routing traffic back through you — meaning no loop is possible.

Key ideas

- **Successor** = the current best path (in routing table). Has the lowest FD.
- **Feasible successor** = a loop-free backup path ready to take over **immediately**. Its RD must be less than the successor's FD.
- If no feasible successor exists, EIGRP runs a **diffusing computation** (queries and replies) to find a new route; this usually takes a few seconds.

Example

Suppose Router A has two paths to a destination:

- Path via B: FD = 100, RD reported by B = 60
- Path via C: FD = 120, RD reported by C = 80

Path via B is the **successor** (lowest FD = 100). Path via C: is C's RD (80) < successor's FD (100)? Yes → C is a **feasible successor** (loop-free backup).

This is why EIGRP is often said to support an "**immediate backup**", unlike OSPF.

EIGRP tables

Table	Purpose
Neighbor table	Stores EIGRP neighbors and their metrics (like RTT, SRTT, RTO).
Topology table	Stores all routes advertised by neighbors, including feasible successors.
Routing table (RIB)	Contains the best routes (successors) selected by DUAL.

OSPF style: **LSDB** → **SPF** → **routing table**. EIGRP style: **Topology table** → **DUAL** → **routing table**.

Auto-summarization and bandwidth management

Auto-summarization

By default, EIGRP **auto-summarizes** routes at **classful boundaries**.

- Example: 192.168.1.0/24 and 192.168.2.0/24 may be summarized to **192.168.0.0/16**.
- This can cause **incorrect routing** if the subnets are in different parts of the network.

To disable it (modern best practice):

```
router eigrp 10
no auto-summary
```

Bandwidth management

EIGRP limits routing traffic to a percentage of the interface bandwidth (default **50%**).

- Prevents routing updates from **flooding low-bandwidth links**.
- On high-speed links, this can be too conservative, so you can adjust it and tune **bandwidth** and **delay** metrics manually.

PDM (Protocol Dependent Module)

EIGRP is designed to support **multiple network-layer protocols**, not just IP. This is possible because of its modular design — each supported protocol gets its own **Protocol Dependent Module (PDM)**.

What PDM does

- Each PDM handles the routing logic for **one specific protocol** (e.g., IPv4, IPv6, IPX).
- The DUAL algorithm and the core EIGRP engine remain the same — only the PDM layer changes depending on the protocol being routed.
- Think of it like a **compartment** or plug-in: one compartment for IPv4, another for IPv6, and so on.

Why it matters

- EIGRP can maintain **separate neighbor tables, topology tables, and routing tables** for each protocol independently.
- A problem in one protocol's routing does not affect the others.
- In modern networks, the most relevant PDMs are **IPv4** and **IPv6 (EIGRPv6)**.

Simple way to think about it

DUAL is the engine. PDM is the gear selector — it tells the engine which protocol's routes to process.

Basic EIGRP commands (Cisco)

```
R(config)# router eigrp 10
R(config-router)# network 192.168.1.0 0.0.0.255
R(config-router)# network 10.0.0.0
```

- **AS number (10)** must be the same on all routers in the EIGRP domain.

Common verification commands

show ip eigrp neighbors	→ EIGRP neighbor table
show ip eigrp topology	→ Topology table (includes feasible successors)
show ip route eigrp	→ EIGRP routes in routing table

13) Loop Prevention Techniques

A **routing loop** happens when a packet goes in circles between routers instead of reaching the destination.

- Wastes **bandwidth**.
- Can cause **black-holes** and slow convergence.
- Common in **distance-vector** protocols (like RIP, EIGRP) if loop-prevention rules are missing.

The goal of all these techniques is: **Stop packets from looping endlessly** and **prevent bad routing info from spreading**.

1. TTL (Time To Live)

- Every IP packet has a **TTL** field (or **Hop-Limit** in IPv6).
- Every time the packet passes a router, **TTL = TTL - 1**.
- When **TTL = 0**, the packet is **dropped** and not forwarded.

Purpose

- **Last-resort safety net**: even if a loop exists, the packet dies after a limited number of hops.
- Does **not fix** the routing bug; it just limits the damage.

2. Split Horizon

In distance-vector protocols, a router **does not advertise a route back out the same interface** it learned it on.

Example:

- Router A tells B about network X.
- Router B will **not** tell A again about X.

Why it helps

- Prevents **back-loops** between two neighboring routers.

- Basic rule in **RIP, EIGRP**, and similar protocols.

3. Route Poisoning

When a route fails (link goes down), the router **advertises it with "infinite" metric**, i.e., unreachable.

Example:

- Network X goes down.
- Router sends: "Distance to X = ∞ " (16 for RIP).
- Neighbors immediately mark it as **unreachable**.

Purpose

- Faster convergence than waiting for hop-count to slowly increase.
- Prevents **count-to-infinity** and routing loops.

4. Poison Reverse

A **stronger version** of Split Horizon.

- Instead of just **not advertising** the route, the router **explicitly advertises it as unreachable** back to the neighbor.

Example:

- Router A tells B about X.
- When X goes down, B **replies to A** with: "Distance to X = ∞ ".

Why it helps

- Neighbor knows **for sure** this path is bad, not just silent.
- More aggressive at preventing loops; used in **RIP with poison-reverse** and EIGRP.

5. Hold-down Timers

When a route disappears, routers **temporarily ignore** new updates for that destination unless the new route comes from the same source or is clearly better.

What it prevents

- **Flapping** (route going up and down).
- Accepting **old/wrong** info from a late-arriving update.

Trade-off

- Slower convergence in some cases, but safer against instability.

6. Triggered (Flash) Updates

Instead of waiting for **periodic updates** every 30 seconds (RIP style), a router **immediately sends an update** when a route changes.

Purpose

- **Faster convergence**: neighbors learn topology changes quickly.
- Reduces chance that others still use an old path, which could cause loops.

7. Sequence Numbers

Some protocols attach a **version number** to each route or message.

- Neighbors accept only **newer** sequence numbers.
- Older numbers are discarded.

Loop-prevention benefit

- Prevents **stale** information from being used after a failure or change.
- Avoids using old routes that might have created loops.

8. Link-State Routing (OSPF style)

In **link-state protocols (OSPF, IS-IS)**:

- Each router builds a **complete map (topology)** of the network using **LSAs**.
- All routers run **Dijkstra's SPF algorithm** on the same topology to find the best path.

Why loops are less likely

- Routers do **not guess** routes from neighbors alone.
- Every router independently computes **loop-free shortest paths** from the shared map.

9. Path-Vector (BGP style)

In **BGP**, each route carries a **full path list** (list of AS numbers or routers).

- When a router sees **its own ID** (AS number) in the path, it **rejects** that route.

Example

- Router R1 advertises a route via AS1 → AS2 → AS3.
- If AS2 tries to send it back to AS1, AS1 sees **its own AS** in the path and **drops** it.

Purpose

- Straight-forward **loop detection**: no router can import a path that includes itself.

Quick conceptual summary

Technique	Main idea	Where it's used
TTL	Eventually kill looping packets	All IP networks
Split Horizon	Do not advertise back the way you learned it	Distance-vector (RIP, EIGRP)
Route Poisoning	Mark failed routes as unreachable immediately	Distance-vector

Technique	Main idea	Where it's used
Poison Reverse	Tell neighbor "this route is dead"	Distance-vector
Hold-down Timer	Wait before accepting new updates for a failed route	Distance-vector
Triggered Updates	Send updates immediately on change	Distance-vector
Sequence Numbers	Use version numbers so only newer routes are accepted	Some protocols
Link-State (OSPF)	Build global map and use Dijkstra for loop-free paths	OSPF, IS-IS
Path-Vector (BGP)	Check path; if it includes me → reject	BGP

Simple way to remember

- **Distance-vector** → Prevent loops with **rules and timers** (split horizon, poisoning, hold-down, triggers).
- **Link-state** → Avoid loops with **complete topology + Dijkstra**.
- **BGP** → Detect loops with **path history** (AS-PATH / router ID list).

14) Load Balancing

Load balancing in OSPF

How it works

- OSPF uses **cost** as its metric.
- If a router has **two or more routes with exactly the same total cost** to a destination, it can install **multiple next-hops** in the routing table.
- Traffic is then **shared across these equal-cost paths** (usually per-flow or per-packet, depending on platform/CEF).

This is called **ECMP – Equal-Cost Multi-Path** routing.

Limits

- On Cisco IOS, by default, OSPF usually uses up to **4 equal-cost paths**.
- You can change this with the `maximum-paths` command under the OSPF process, up to higher values (often up to 16 or 32, depending on platform/IOS).

Example:

```
router ospf 1
maximum-paths 4
```

- OSPF does **not** support unequal-cost load balancing in classic implementations: Only routes with **identical cost** are used together.

Load balancing in EIGRP

EIGRP is more flexible: it supports **equal-cost** and **unequal-cost** load balancing.

1. Equal-cost load balancing (default)

- If EIGRP learns multiple routes to a destination with the **same composite metric**, it can install multiple next-hops and share traffic across them.
- By default, Cisco routers use up to **4 equal-cost paths**, controlled by **maximum-paths**.

Example:

```
router eigrp 100
maximum-paths 4      ! Can be raised up to 16 or 32 depending on platform
```

2. Unequal-cost load balancing (variance)

This is the big EIGRP advantage over OSPF.

- EIGRP can also use **paths with different metrics** if they are "**good enough**" compared to the best path.
- This is controlled by the **variance** command (a multiplier).

Logic (simplified):

1. EIGRP finds the **best path** (successor) with metric = **M**.
2. You configure **variance n**.
3. Any **feasible successor** (loop-free backup) with metric $\leq n \times M$ can also be used for forwarding.

Example:

```
router eigrp 100
variance 2
maximum-paths 4
```

- Here, EIGRP will use:
 - The best path (metric M), and
 - Any feasible successors with metric $\leq 2 \times M$, for **unequal-cost load sharing**.

Important:

- Only **feasible successors** are used for unequal-cost load balancing (must pass feasibility condition, so they are loop-free).

Traffic-share ratios

When EIGRP does unequal-cost load balancing, it does **not** split traffic equally. Instead, it sends **more traffic through the better path** and less through the worse one, in proportion to their metrics.

The traffic share is calculated as:

Traffic share $\propto 1 / \text{metric}$ — the path with the lower metric carries more traffic.

Example:

Suppose three paths have metrics of 100, 200, and 500:

Path	Metric	Inverse (1/metric)	Traffic share (simplified)
Path A (successor)	100	$1/100 = 10$	10 parts
Path B (feasible successor)	200	$1/200 = 5$	5 parts
Path C (feasible successor)	500	$1/500 = 2$	2 parts

So the ratio is **10 : 5 : 2** — Path A carries roughly 10 packets for every 5 sent via Path B and 2 via Path C.

This is called **proportional load sharing** — faster/better paths carry more of the load automatically.

Quick comparison

Feature	OSPF	EIGRP
Metric	Cost (bandwidth-based)	Composite (bandwidth, delay, etc.)
Equal-cost load balancing	Yes (ECMP, maximum-paths)	Yes (maximum-paths)
Unequal-cost load balancing	No (classic OSPF)	Yes (variance with feasible successors)

Mental shortcut:

- **OSPF** → "Equal only" (ECMP over same-cost paths).
- **EIGRP** → "Equal + Unequal" (tune with **maximum-paths** and **variance**).

15) RIP vs OSPF vs EIGRP — Quick Comparison

Feature	RIP	OSPF	EIGRP
Protocol type	Distance vector	Link-state	Hybrid (advanced distance vector)
Metric	Hop count (max 15)	Cost (based on bandwidth)	Composite (bandwidth + delay by default)
Administrative Distance	120	110	90 (internal) / 170 (external)
Convergence speed	Slow	Fast	Very fast (DUAL + feasible successors)

Feature	RIP	OSPF	EIGRP
Max hops / scale	15 hops hard limit; not suitable for large networks	No hop limit; scales well using areas	No hop limit; scales to large enterprise networks
Authentication	RIPv2 supports MD5; RIPv1 has none	MD5 (OSPFv2); OSPFv3 uses IPsec	MD5
Update type	Periodic full-table every 30 seconds	Triggered partial updates on change	Triggered partial updates on change
Open standard	Yes (RFC 1058 / RFC 2453)	Yes (RFC 2328)	Cisco proprietary (later published as RFC 7868)
Load balancing	Equal-cost only	Equal-cost only (ECMP)	Equal-cost and unequal-cost (via variance)

Simple memory shortcut

- **RIP** → Simple, slow, small networks only. Hop count is the only metric. Good for learning; rarely used in production today.
- **OSPF** → Open standard, cost-based metric, scales well with areas. The most common choice in real-world networks.
- **EIGRP** → Cisco-focused, fastest convergence via DUAL, flexible load balancing with **variance**. Best in Cisco-only environments.

1. VLAN Concept

- **VLAN (Virtual Local Area Network)** = a **logical grouping** of networking devices, regardless of their physical location.
- Creating a VLAN **breaks one large broadcast domain into multiple smaller broadcast domains**.
- Think of a VLAN like a **subnet**: just as two different subnets cannot communicate without a router, **two different VLANs also require a router (or Layer 3 device) to communicate**.
- Devices in the same VLAN can communicate directly (Layer 2); devices in different VLANs need Layer 3 routing.

✦ **Exam Trap:** VLAN = Broadcast Domain = Subnet (conceptually mapped 1:1 in most designs).

🔑 **Key Point — Trunk Link:** A **trunk link carries traffic for multiple VLANs over a single physical connection** (one cable, many VLANs — each frame tagged with its VLAN ID so the far end can sort it back out). An **access link**, by contrast, carries traffic for **only one VLAN**.

1.1 VLAN ID (VLAN Number) and VLAN Range

Every VLAN is identified by a unique **VLAN ID** (a number) so switches can distinguish one VLAN's traffic from another's, especially over a trunk link.

- ◆ VLAN ID Field

- Carried inside the **802.1Q tag** in the Ethernet frame (12-bit field).
- 12 bits → theoretical range = **0 – 4095** → but 0 and 4095 are reserved, so usable range = **1 – 4094**.

◆ VLAN ID Ranges (Cisco Switches)

Range	Type	Notes
0	Reserved	Not used (priority tagging only, no VLAN)
1	Default VLAN	Exists automatically on every switch; cannot be deleted or renamed; carries CDP, VTP, PAgP, STP traffic by default
2 – 1001	Normal Range	User-configurable VLANs; stored in vlan.dat ; advertised via VTP
1002 – 1005	Reserved (legacy)	Auto-created for legacy Token Ring / FDDI; cannot be deleted
1006 – 4094	Extended Range	User-configurable; NOT advertised by VTP (in VTP versions 1 & 2); stored in running-config, not vlan.dat
4095	Reserved	Used internally, not assignable

✅ **Memory Trick: 1 = Default | 2–1001 = Normal | 1002–1005 = Legacy Reserved | 1006–4094 = Extended | 4095 = Reserved**

◆ Normal Range vs Extended Range VLANs

Parameter	Normal Range VLAN	Extended Range VLAN
VLAN ID	1 – 1001	1006 – 4094
Stored In	vlan.dat (flash memory)	running-config only
VTP Advertisement	Yes (VTPv1/v2)	No (VTPv1/v2) — VTPv3 does support it
Typical Use	Most enterprise deployments	Service provider / very large networks

✳️ **Exam Trap:** VLAN 1 is the **default VLAN** and also commonly the **native VLAN** — both can be changed except VLAN 1 itself can never be deleted.

2. Benefits / Advantages of VLAN

Benefit	Explanation
Solves broadcast problem	Limits broadcast traffic to within the VLAN only
Reduces broadcast domain size	Smaller domains = less congestion, better performance
Additional layer of security	Sensitive systems can be isolated into their own VLAN

Benefit	Explanation
Easier device management	Devices grouped logically, easier to administer centrally
Logical grouping by function	Group by department/role instead of physical location (e.g., all HR PCs in one VLAN regardless of floor)

✅ **Memory Trick: S-R-A-M-L** → Solve broadcast, Reduce domain, Add security, Manage easily, Logical grouping

3. VLAN Port Assignment / Connection Types

When configuring VLANs on a switch port, we must define the **connection type**. Switches support **two types** of VLAN connections:

1. **Access Link**
2. **Trunk Link**

◆ Access Link

- Connects a switch port to an end device with a **standard Ethernet NIC**.
- Standard NICs only understand **IEEE 802.3 / Ethernet II** frames (no VLAN tag).
- An access port can be assigned to **only ONE VLAN** at a time.
- All devices connected via that access port belong to the **same broadcast domain (VLAN)**.

Example:

20 users connected to a hub → hub connected to a switch access port → all 20 users are in the **same VLAN**. To put 10 users in a different VLAN, you'd need a **separate hub** connected to a **different access port**.

◆ Trunk Link

- Connects a switch port to a device **capable of understanding multiple VLANs** (usually **switch-to-switch** or **switch-to-router**).
- Allows VLAN information to be carried **across the network** — this is how a VLAN can "span" multiple switches.
- Requires the original Ethernet frame to be **modified** to carry VLAN membership information (**tagging**).

◆ Access vs Trunk — Comparison Table

Parameter	Access Link	Trunk Link
Connects to	End devices (PC, printer, standard NIC)	Switches, routers
VLANs Supported per Port	Only 1 VLAN	Multiple VLANs

Parameter	Access Link	Trunk Link
Frame Type	Standard Ethernet (untagged)	Tagged (ISL / 802.1Q)
Purpose	Connect end-user devices	Carry VLAN traffic between network devices
Requires Tagging Protocol	No	Yes (Dot1q or ISL)

4. VLAN Tagging — IEEE 802.1Q

◆ What is Tagging?

- In trunking, a **separate logical connection** is created for each VLAN (instead of one physical connection per VLAN).
- The switch **adds the source port's VLAN identifier** to the Ethernet frame — this is called **tagging**.
- This tag tells the receiving switch **which VLAN the frame originated from**, allowing intelligent forwarding decisions based on **both** the destination MAC address **and** the source VLAN ID.
- Tagging is performed in **hardware** by **ASICs (Application-Specific Integrated Circuits)** for speed.

✦ Since the frame is modified, a **standard NIC will NOT understand tagged frames** and will typically **drop them**. Both ends of a trunk link **must support and be configured with the same trunking protocol**.

◆ Trunking Protocols (2 types)

Protocol	Full Form	Vendor
ISL	Inter-Switch Link	Cisco- proprietary
Dot1q	IEEE 802.1Q	Open industry standard

◆ IEEE 802.1Q Details

- Industry-standard trunking protocol (works across vendors, unlike ISL).
- Inserts a **4-byte tag** into the Ethernet frame header, containing:
 - **TPID (Tag Protocol Identifier)** — identifies the frame as 802.1Q tagged
 - **VLAN ID (12 bits)** — supports VLAN IDs from **1 to 4094**
 - **Priority bits (3 bits)** — for QoS (CoS — Class of Service)
- **Native VLAN concept:** 802.1Q does **NOT tag** frames belonging to the **native VLAN** (default VLAN 1 unless changed) — these are sent **untagged** across the trunk.

✅ Exam Trap:

- ISL = Cisco proprietary, **tags all frames including native VLAN**, now largely obsolete.
- 802.1Q = Industry standard, **does NOT tag native VLAN traffic**, widely used today.

5. Inter-VLAN Routing

Since different VLANs are like different subnets, communication **between VLANs requires Layer 3 routing**.

There are **three options**:

1. **Router with one physical interface per VLAN** (traditional method — typically NOT used; wastes physical ports)
2. **Router-on-a-Stick** (single router interface, VLAN trunk to switch)
3. **Layer 3 Switch** (Switched Virtual Interfaces — SVIs, most common in modern networks)

◆ Option 1: One Router Interface per VLAN

- Requires a **dedicated physical router interface for each VLAN**.
- **Disadvantage**: Not scalable — limited by number of physical router ports; wasteful and expensive.

◆ Option 2: Router-on-a-Stick

- A configuration that allows routing of traffic **between VLANs using a single physical router interface**.
- The router has **one physical interface**, but that interface is divided into multiple **logical sub-interfaces** — one per VLAN.
- Router connects to the switch via a **VLAN trunk** (802.1Q).
- Each subnet's hosts use the router's sub-interface IP (in that VLAN) as their **default gateway**.

Configuration Steps (Cisco):

```
! Step 1: Create a sub-interface for each VLAN
interface <type><number>.<subinterface-number>

! Step 2: Enable 802.1Q trunking and associate the sub-interface with a VLAN
encapsulation dot1q <vlan_id>

! Step 3: Assign IP address to the sub-interface
ip address <address> <mask>
```

Example:

```
interface fastethernet0/0.10
  encapsulation dot1q 10
  ip address 192.168.10.1 255.255.255.0

interface fastethernet0/0.20
  encapsulation dot1q 20
  ip address 192.168.20.1 255.255.255.0
```

✅ **Exam Trap**: Router-on-a-stick needs only **ONE physical interface** but **multiple logical sub-interfaces**, each tied to a VLAN via **encapsulation dot1q**.

◆ Option 3: Layer 3 Switch (Preferred in Modern Networks)

- A switch capable of performing **both Layer 2 switching AND Layer 3 routing**.

- Uses **SVI (Switched Virtual Interface)** — a virtual interface representing a VLAN, assigned an IP address, acting as the default gateway for that VLAN.
- **Faster** than router-on-a-stick (hardware-based switching/routing, no trunk bottleneck).
- Most commonly used method in **enterprise networks today**.

Basic Configuration Concept:

```
interface vlan 10
ip address 192.168.10.1 255.255.255.0
no shutdown
```

◆ Comparison of Inter-VLAN Routing Methods

Method	Physical Interfaces Needed	Speed	Scalability	Common Usage
One interface per VLAN	Many (1 per VLAN)	Fast (dedicated)	Poor	Rare / legacy
Router-on-a-Stick	1 (with sub-interfaces)	Slower (bottleneck at trunk)	Medium	Small/medium networks, labs
Layer 3 Switch (SVI)	0 (virtual interfaces)	Fastest (hardware-based)	Excellent	Modern enterprise networks

6. VTP (VLAN Trunk Protocol)

◆ What is VTP?

- **VTP (VLAN Trunk Protocol)** reduces administrative overhead in a switched network.
- When a new VLAN is configured on **one VTP server**, that VLAN information is **automatically distributed to all switches** in the same VTP domain.
- Eliminates the need to **manually configure the same VLAN on every switch**.
- **VTP is Cisco-proprietary**, available mainly on Cisco Catalyst series switches.

◆ VTP Features

- Advertises VLAN configuration information across the network
- Maintains **VLAN configuration consistency** throughout a common administrative domain
- Sends advertisements **only on trunk ports**

◆ VTP Modes (3 Modes — Very Important)

Mode	Can Create/Modify/Delete VLANs?	Forwards VTP Advertisements?	Saves VLAN Info in NVRAM?
Server	✓ Yes	✓ Yes	✓ Yes

Mode	Can Create/Modify/Delete VLANs?	Forwards VTP Advertisements?	Saves VLAN Info in NVRAM?
Client	✗ No	✓ Yes	✗ No (learns from Server)
Transparent	✓ Yes (local only, not advertised)	✓ Yes (forwards, doesn't process)	✓ Yes (local database only)

Details:

- **Server Mode (default mode):**

- Full control — can create, modify, and delete VLANs.
- Advertises VLAN info to other switches in the domain.
- Stores VLAN configuration in NVRAM (persists across reboot).

- **Client Mode:**

- **Cannot** create/modify/delete VLANs locally.
- Only **receives and forwards** VTP advertisements from servers.
- Does **not** save VLAN info to NVRAM — relies on Server for updates on every boot.

- **Transparent Mode:**

- Can create/modify/delete VLANs, but changes are **local only** — NOT advertised to other switches.
- **Forwards** VTP advertisements it receives from other switches (acts as a pass-through) but does **not process/act on them**.
- Saves its own VLAN configuration locally in NVRAM.

✓ **Memory Trick: Server = Full Control + Advertises Client = No Control, Just Listens Transparent = Local Control Only, Just Relays**

♦ VTP Operation

- VTP advertisements are sent as **multicast frames**.
- VTP **servers and clients** synchronize using the **latest Configuration Revision Number** — higher revision number = more recent = gets adopted.
- Advertisements are sent **every 5 minutes**, or immediately when a change occurs (triggered).

⚠ **Critical Danger (Common Real-World/Exam Scenario):** If a **new switch with a higher VTP revision number** (even with wrong/empty VLAN data) is added to the domain, it can **overwrite the VLAN database** of the entire network — causing major outages!

♦ VTP Configuration Guidelines

Configuration items required for VTP setup:

- **VTP Domain Name** — must match across all switches in the domain to communicate
- **VTP Mode** — Server mode is default
- **VTP Pruning** — restricts unnecessary VLAN traffic on trunk links where not needed (saves bandwidth)

- **VTP Password** — secures the domain from unauthorized VTP changes

◆ Best Practice When Adding a New Switch to an Existing VTP Domain

To avoid accidentally overwriting the domain's VLAN database:

1. **Add the new switch in Client mode first** → allows it to safely learn/synchronize the latest VLAN information from the domain without risk of overwriting anything.
2. Once synced, **convert it to Server mode** if it needs to make changes.

Alternative approach:

1. Add all new configurations to the switch while it is in **Transparent mode** first.
2. Thoroughly **check/verify the configuration**.
3. Then **convert it to Server mode** — this prevents the switch from propagating incorrect/incomplete VLAN information to the rest of the domain during setup.

✅ **Exam Trap:** Never add a brand-new switch directly in **Server mode** with default settings blindly — the real danger is adding a switch that was **previously configured elsewhere** with a **higher revision number** — it can silently wipe out the domain's VLANs.

6.1 What is NAT?

Network Address Translation (NAT) is the process where a network device — usually a **router or firewall** — translates a **private IP address** into a **public IP address** (and vice versa) as traffic crosses the boundary between a private network and the outside network (typically the Internet).

- NAT is typically performed on a **border router/firewall** sitting between the inside (private) network and the outside (public) network.
- Assigns a **public address** to a computer (or group of computers) inside a private network.

6.2 Why NAT is Used — IPv4 Exhaustion

- IPv4 has only **~4.3 billion** addresses total — nowhere near enough for every device on Earth to have a unique public IP.
- NAT was introduced as a **short-term fix** to slow down the exhaustion of public IPv4 addresses (long-term fix = IPv6).

◆ Main Reasons NAT is Used

Reason	Explanation
IPv4 Address Conservation	Limits the number of public IPs an organization must purchase/use
Economy	Public IPs cost money (leased from ISP); NAT lets many private hosts share few public IPs
Security	Hides internal IP addressing scheme from the outside world (internal topology not exposed)

Reason	Explanation
Flexibility	Internal network can be renumbered/restructured without affecting external-facing addresses
Multihoming/ISP independence	Internal addressing doesn't need to change even if ISP or public IP changes

✦ **Exam Trap:** NAT is fundamentally an **IPv4 exhaustion workaround** — it does NOT eliminate the need for public IPs, it just drastically **reduces how many are needed**.

6.3 Private IP Address Ranges (RFC 1918)

These ranges are **reserved for private/internal use only** — they are **not routable on the public Internet**. Any device using these must go through NAT to reach the internet.

Class	Private IP Range	Total Addresses	CIDR
Class A	10.0.0.0 – 10.255.255.255	~16.7 million	10.0.0.0/8
Class B	172.16.0.0 – 172.31.255.255	~1 million	172.16.0.0/12
Class C	192.168.0.0 – 192.168.255.255	65,536	192.168.0.0/16

✅ **Memory Trick:** **10.x.x.x** → **172.16.x–172.31.x** → **192.168.x.x** (Class A = biggest range for large enterprises; Class C = smallest, most common in homes/small offices)

✦ **Exam Trap:** Common mistake — thinking **all** of **172.x.x.x** is private. Only **172.16.0.0 to 172.31.255.255** is private; e.g., **172.32.0.0** is a **public** address.

◆ Other Special/Reserved Ranges (Bonus)

Range	Purpose
127.0.0.0/8	Loopback (localhost)
169.254.0.0/16	APIPA (Automatic Private IP Addressing)

6.4 NAT Addressing Terminology (Very Important)

NAT uses **4 specific address terms**, based on the combination of **Inside/Outside** and **Local/Global**:

Term	Meaning
Inside Local	The actual private IP address assigned to a host inside the enterprise network (the "real" internal address)
Inside Global	The public/translated IP address that represents an inside host to the outside network (what the internet sees)

Term	Meaning
Outside Global	The actual real public IP address of a host that resides outside the enterprise (e.g., a web server on the internet)
Outside Local	The address used to represent an outside host as seen from inside the private network (rarely different from Outside Global unless doing NAT on both sides)

◆ Simple Way to Remember

- **"Inside"** = belongs to your private network
- **"Outside"** = belongs to the internet/external network
- **"Local"** = how the address appears **inside** the private network
- **"Global"** = how the address appears **outside**, on the public network

Direction of Translation:

- **Inside → Outside:** Source IP changes from **Inside Local** → **Inside Global**
- **Outside → Inside:** Destination IP changes from **Inside Global** → **Inside Local**

✓ **Memory Trick:** **Inside Local** = "my real private IP" **Inside Global** = "how I look to the outside world"
Outside Global = "the real IP of the outside server" **Outside Local** = "how the outside server looks to me"
 (usually same as Outside Global unless double NAT)

6.5 NAT Table Concept

The **NAT Table** (also called the **NAT Translation Table**) is a table maintained by the NAT router/firewall that keeps track of the **mappings between inside local and inside global addresses** (and ports, in PAT).

◆ What the NAT Table Stores

- Inside Local IP (+ port, for PAT)
- Inside Global IP (+ port, for PAT)
- Outside Global IP (+ port, if applicable)
- Protocol (TCP/UDP)
- Idle timer (how long the entry stays before being removed if unused)

◆ Example NAT Table (PAT/Overload)

Inside Local	Inside Global	Outside Global	Protocol
192.168.1.10:5001	203.0.113.5:10001	8.8.8.8:80	TCP
192.168.1.11:5002	203.0.113.5:10002	142.250.1.1:443	TCP
192.168.1.12:5003	203.0.113.5:10003	93.184.216.34:80	TCP

- When a return packet arrives at the router, it checks the NAT table to see **which inside host** the packet belongs to, based on the **destination port**, and forwards it accordingly.
- **Static NAT** entries → **permanent** in the table (manually configured, never age out).

- **Dynamic NAT / PAT** entries → **temporary**, stay in the table only as long as traffic flows; removed after an **idle timeout**.

6.6 Types of NAT

There are **4 main types**:

1. **Static NAT**
2. **Dynamic NAT**
3. **PAT (Port Address Translation) / NAT Overload**
4. **NAT64** (IPv6 ↔ IPv4 transition mechanism)

◆ Quick Comparison Table

Type	Mapping	Public IPs Used	Use Case
Static NAT	1-to-1, permanent	One public IP per private IP	Server that must be reachable from outside
Dynamic NAT	Many-to-many (pool)	Pool of public IPs	Multiple hosts; pool can run out
PAT / NAT Overload	Many-to-1 (using ports)	One public IP, many ports	Homes, businesses (most common NAT type)
NAT64	IPv6 ↔ IPv4	NAT64 prefix	IPv4-to-IPv6 migration/transition

6.7 Static NAT

- Defines a **one-to-one, permanent mapping** between one private (inside local) IP and one public (inside global) IP.
- Mapping includes **destination IP translation** in one direction and **source IP translation** in the reverse direction.
- **Manually configured** — does NOT age out or change automatically.
- **A public IP address must be allocated for every single private IP** that needs static NAT — no address pools involved.
- Allows connections to be **originated from either side** (inside → outside OR outside → inside) — this is important because it's the only NAT type that reliably supports **inbound connections initiated from the internet**.

◆ Static NAT Diagram

Static NAT:
Inside: 192.168.1.10 → Router → Outside: 203.0.113.5

◆ Use Case

- **Servers that must be reachable from the internet** — e.g., a web server, mail server, or any host that external users need to initiate connections to.

✅ **Exam Trap:** Static NAT does **NOT conserve IP addresses** — it's still 1-to-1 — its purpose is **reachability/accessibility**, not conservation.

6.8 Dynamic NAT

- Also creates a **one-to-one mapping** between inside local and inside global addresses, BUT the mapping is chosen **dynamically** from a **pool of available public IPs**, rather than being manually fixed.
- Router defines:
 - A **pool of possible inside global (public) addresses**
 - **Criteria** (via ACL) for which inside local addresses should be translated
- The dynamic entry stays in the NAT table **only as long as traffic is flowing occasionally** — it **ages out** after a period of inactivity.

♦ Dynamic NAT Diagram

```
Dynamic NAT:
192.168.1.10 → 203.0.113.5
192.168.1.11 → 203.0.113.6
192.168.1.12 → 203.0.113.7 } — NAT Pool
```

♦ Key Limitation

- If the **number of inside hosts needing translation exceeds the pool size**, some hosts will **fail to get a public IP** and be unable to reach the internet until an address frees up.

✅ **Exam Trap:** Dynamic NAT is still **1-to-1 at any given moment** — it just **automates address assignment** from a pool. It does NOT let multiple hosts share ONE IP simultaneously (that's PAT).

6.9 PAT (Port Address Translation) / NAT Overload

- Also called **NAT Overloading** or **NAPT (Network Address Port Translation)**.
- A modified form of Dynamic NAT where the **number of inside local addresses is greater than the number of inside global addresses**.
- Typically, **just ONE single public IP** provides internet access for **ALL inside hosts**.
- Distinguishes between multiple internal hosts sharing the same public IP by using **different source port numbers** for each translated connection.
- **The only NAT type that actually conserves IP addresses** — because many private hosts share just one public IP.
- **Most popular/common form of NAT** used today (homes, small businesses, most SOHO routers).

♦ PAT Diagram

PAT (NAT Overload):

```
192.168.1.10 → 203.0.113.5 : 10001
192.168.1.11 → 203.0.113.5 : 10002
192.168.1.12 → 203.0.113.5 : 10003
```

} — Same public IP, different ports

◆ Why PAT Works

- TCP/UDP has **65,536 possible port numbers** per IP address.
- By mapping each internal host's connection to a **unique port** on the single shared public IP, the router can track and correctly route return traffic back to the right internal host.

✅ **Exam Trap:** PAT = **Many-to-One** using **ports** to differentiate; this is what makes home routers work with just 1 public IP for an entire household of devices.

6.10 NAT Types — Full Side-by-Side Comparison

Parameter	Static NAT	Dynamic NAT	PAT / NAT Overload
Mapping Type	1-to-1 (fixed)	1-to-1 (from pool)	Many-to-1 (via ports)
Public IP Requirement	1 per private IP	Pool of public IPs	Just 1 public IP (typically)
Conserves IP Addresses?	❌ No	⚠️ Partially	✅ Yes (best conservation)
Connection Origination	Either direction	Inside → Outside only	Inside → Outside only
Table Entry	Permanent	Temporary (ages out)	Temporary (ages out)
Common Use Case	Public-facing servers	Medium orgs with IP pool	Homes, small/large businesses
Popularity	Less common	Less common today	Most widely used

6.11 NAT64

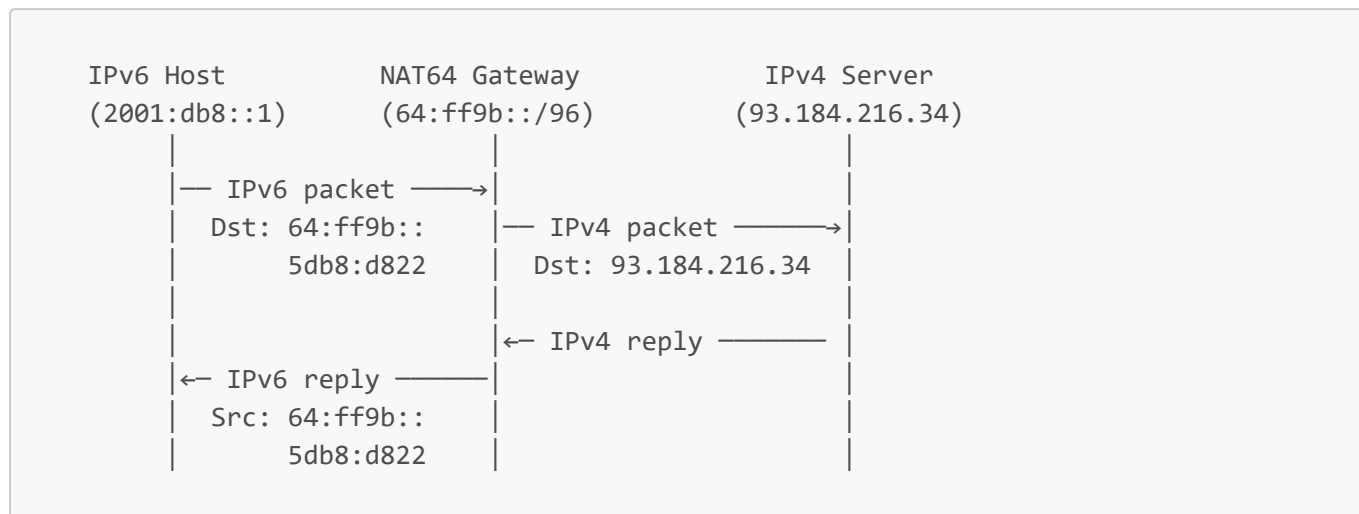
NAT64 allows **IPv6-only hosts** to communicate with **IPv4-only servers**, by translating between IPv6 and IPv4 addresses at the network boundary. Works **alongside DNS64**.

◆ How NAT64 Works (Step-by-Step)

1. An **IPv6-only host** queries **DNS64** for an IPv4-only domain (e.g., `example.com`).
2. DNS64 finds **only an IPv4 A record** (e.g., `93.184.216.34`). Since no AAAA (IPv6) record exists, DNS64 **synthesizes** one by embedding the IPv4 address into the **NAT64 prefix** (`64:ff9b::/96`):
 - Synthesized AAAA: `64:ff9b::5db8:d822`
3. The host sends an **IPv6 packet** to `64:ff9b::5db8:d822`.
4. The **NAT64 gateway** intercepts the packet, **extracts the embedded IPv4 destination** (`93.184.216.34`), translates **IPv6 → IPv4**, and forwards it to the real IPv4 server.

5. The **IPv4 server replies** → the NAT64 gateway translates **IPv4** → **IPv6** → sends the reply back to the original IPv6 host.

◆ NAT64 Flow Diagram



◆ NAT64 vs Traditional NAT

Feature	Traditional NAT	NAT64
Translates	Private IPv4 ↔ Public IPv4	IPv6 ↔ IPv4
Purpose	Address conservation	Protocol transition
Needs DNS?	✗ No	✓ Yes (DNS64 required)
Direction	IPv4 client → IPv4 server	IPv6 client → IPv4 server
Use Case	Home/office internet sharing	IPv4-to-IPv6 migration

◆ NAT64 Key Terms

Term	Meaning
NAT64	Translates between IPv6 and IPv4
DNS64	Synthesizes AAAA records from existing IPv4 A records
NAT64 Prefix	64:ff9b::/96 — the last 32 bits of this prefix hold the embedded IPv4 address
Stateful NAT64	Many-to-one mapping (like PAT) — many IPv6 hosts share one IPv4 address
Stateless NAT64	One-to-one mapping — requires a dedicated block of IPv4 addresses

✓ **Exam Trap:** NAT64 is fundamentally different in **purpose** from traditional NAT — traditional NAT conserves addresses **within the same protocol (IPv4↔IPv4)**; NAT64 exists purely to let **two different protocol versions (v6 and v4) talk to each other** during the IPv6 migration period.

0. Full Forms / Abbreviations

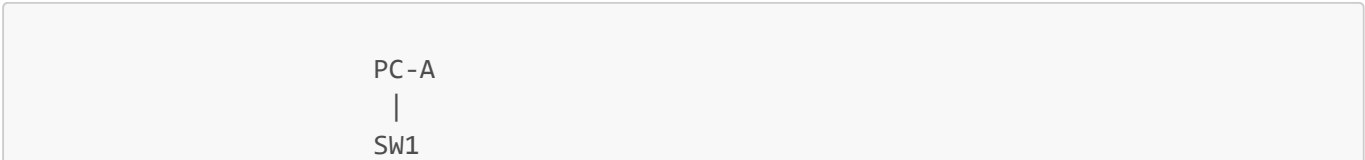
Abbreviation	Full Form
STP	Spanning Tree Protocol
RSTP	Rapid Spanning Tree Protocol
MSTP	Multiple Spanning Tree Protocol
PVST+	Per-VLAN Spanning Tree Plus
RPVST+	Rapid Per-VLAN Spanning Tree Plus
BPDU	Bridge Protocol Data Unit
TCN (BPDU)	Topology Change Notification (BPDU)
BID	Bridge ID
RP	Root Port
DP	Designated Port
TTL	Time To Live
ARP	Address Resolution Protocol
MAC	Media Access Control (address)
VLAN	Virtual Local Area Network
IEEE	Institute of Electrical and Electronics Engineers

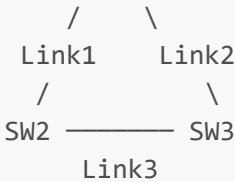
1. Why STP Is Needed — The Layer 2 Loop Problem

Real-world networks deliberately use **redundant links** between switches for fault tolerance (a backup path if one link fails). The problem: **Ethernet frames have no TTL** (unlike IP packets) — nothing stops a frame from circulating forever if a loop exists. Without a loop-prevention mechanism, three problems happen **simultaneously**:

#	Problem	What Happens
①	Broadcast Storm	A single ARP request loops endlessly → CPU on all switches hits 100% → network collapses completely
②	MAC Table Instability (Flapping)	The same source MAC appears to arrive from multiple ports in rapid succession → switches keep re-learning it → forwarding becomes unreliable
③	Duplicate Frame Delivery	The same unicast frame arrives at the destination multiple times → breaks upper-layer protocols like TCP

1.1 Without STP — Loop Present

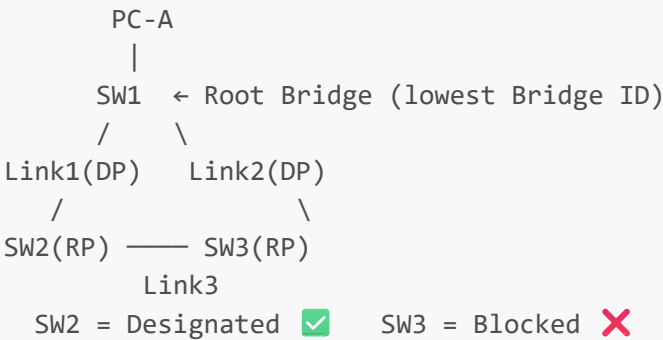




Step 1: PC-A sends a broadcast → SW1 floods it out BOTH Link1 and Link2.
Step 2: SW2 (via Link1) floods it onward via Link3.
SW3 (via Link2) floods it back via Link3.
Step 3: Both switches receive the SAME broadcast again → and flood it again → infinite loop.

RESULT: BROADCAST STORM – network CRASHES

1.2 With STP — Loop Broken



Loop BROKEN – one port blocked, backup path ready to activate automatically if SW2 fails

1.3 Problem → STP Solution

Problem Without STP	How STP Solves It
Broadcast storms crash the network	Blocks redundant ports to eliminate loops
MAC table instability / flapping	Stable topology → stable, predictable MAC learning
Duplicate frame delivery	Enforces a single forwarding path per network segment
Ethernet has no Layer-2 TTL	STP itself acts as the loop-prevention mechanism

2. What Is STP, In Plain Terms?

- STP (**IEEE 802.1D**) was created to **prevent Layer 2 loops** while still keeping redundant physical links available as backups.
- STP ensures there is only **one logical (active) path** between any two points on the network, by intentionally **blocking** redundant paths that could otherwise cause a loop.
- A **blocked port** stops **user data** from entering or leaving it — but it still sends/receives **BPDU frames** (the control messages STP itself uses to detect and manage loops).
- The physical cabling/redundant paths still **exist** — they're just administratively/logically disabled. If the active path fails, STP **recalculates** and automatically unblocks the standby port to restore connectivity.

Key idea:

Physical topology = has loops (redundant links, by design)

Logical topology = loop-free (STP blocks the redundant ports)

3. How Switches Discover Loops — BPDUs

- Switches discover the presence of loops (and each other) by sending **probes** into the network.
- These probes are called **BPDUs** — **Bridge Protocol Data Units**.
- Each BPDU carries specific information identifying the sending switch (its Bridge ID, path cost, etc. — see §4.2).
- Every switch **multicasts** BPDU probes **every 2 seconds** (the Hello Time, §5.1).
- **Loop detection logic:** if a switch ever receives **its own BPDU back**, that confirms a loop exists in the network.

BPDUs serve **two purposes**:

1. **Loop detection** — as above.
2. **Root Bridge election** — BPDUs carry the Bridge ID used to elect the Root Bridge (§4).

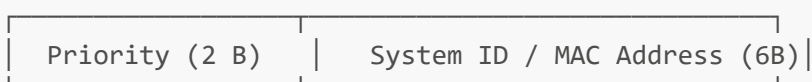
Once the Root Bridge is elected, every other switch calculates the **best (lowest-cost) path** to reach it, and **blocks** any redundant/extra links (based on port cost — see §6). Those blocked links become **active automatically** only if the currently active link or port fails.

4. Root Bridge Election

4.1 Bridge ID (BID)

Every switch has a **Bridge ID**, which is what gets compared during root bridge election.

Classic 802.1D BID structure:



Extended System ID (used in PVST+ / Rapid-PVST+, which run one STP instance per VLAN):

Priority field = Bridge Priority + VLAN ID
Example: $32768 + 10 = 32778$

4.2 Election Rule

Root Bridge Election Rule:

1. LOWEST Bridge ID (BID) wins.
2. If there's a tie on Priority → LOWEST MAC address wins.

Example — three switches with equal priority (32768):

32768.1111.1111.1111 ← Lowest MAC → becomes ROOT BRIDGE
32768.2222.2222.2222
32768.3333.3333.3333

Since all three share the same default priority (32768), the tie-breaker is the MAC address — **1111.1111.1111** is numerically lowest, so that switch wins the election and becomes the **Root Bridge**.

BPDU (Bridge ID) contents used in the election:

- Bridge Priority
- MAC Address of the switch

5. STP Timers & BPDU Types

5.1 Key STP Timers

Timer	Default Value	Purpose
Hello Time	2 s	How often the Root Bridge sends BPDUs
Max Age	20 s	How long a switch waits before assuming the Root Bridge is gone (BPDU not heard)
Forward Delay	15 s	Time spent in the Listening state + time spent in the Learning state (15s each)

💡 Total 802.1D Convergence Time:

$\text{Max Age} + (2 \times \text{Forward Delay}) = 20 + (2 \times 15) = 50 \text{ seconds}$

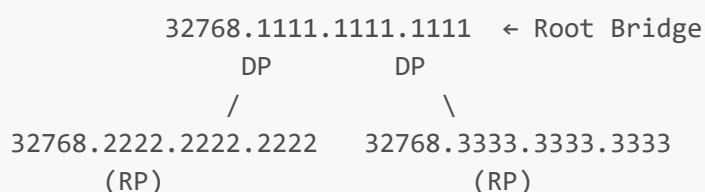
5.2 BPDU Types

Type	Purpose
Configuration BPDU	Carries the Root BID, path cost, sender's BID, port ID, and timers. Sent every 2 seconds (Hello Time), originated by the Root Bridge and relayed downstream
TCN BPDU (Topology Change Notification)	Sent when a port changes state (e.g., link up/down); triggers the rest of the network to update its topology info faster

6. Port Roles

Once the Root Bridge is elected, every other ("non-root") switch assigns roles to its own ports:

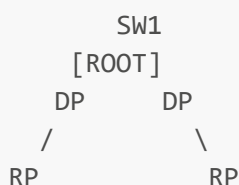
Port Role	Meaning
Root Port (RP)	The port used to reach the Root Bridge via the lowest-cost path — exactly one per non-root switch
Designated Port (DP)	The forwarding port for a given network segment/link — exactly one per link (both ends of a link can't both be DP)
Blocking / Non-Designated Port	Neither root nor designated — kept in blocking state to prevent a loop; does not forward user data

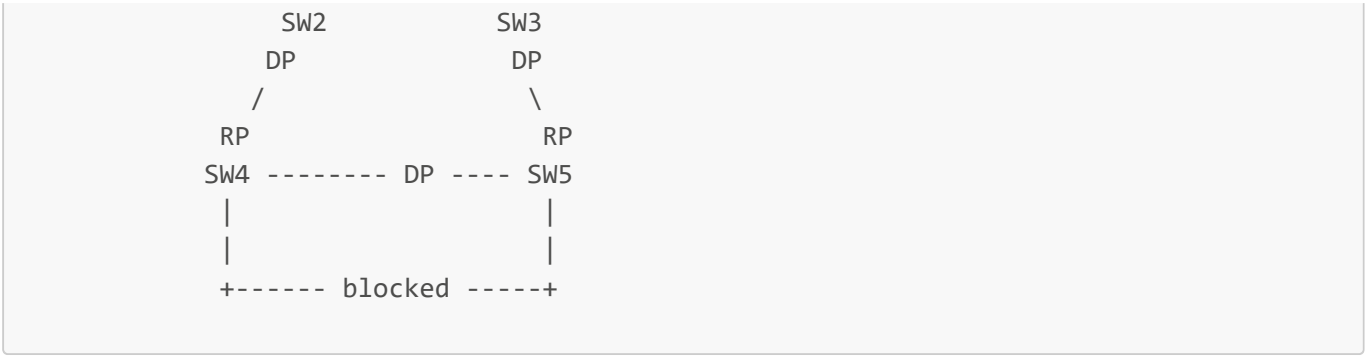


- **DP (Designated Port):** forwards traffic for its segment — the Root Bridge's ports are always DP.
- **RP (Root Port):** each non-root switch picks the port with the best path back to the Root as its Root Port.
- Any additional redundant port that is neither RP nor DP goes into **Blocking** state to eliminate the loop, but stays ready to take over if needed.

6.1 Final Simplified View — 5-Switch Example

A larger topology follows the exact same logic — every non-root switch has one Root Port, every link has one Designated Port, and any leftover redundant port is blocked:





Reading this diagram:

- **SW1** = Root Bridge → both its ports are **DP** (Root Bridge ports are always Designated).
- **SW2** and **SW3** each use their uplink to SW1 as their **Root Port (RP)** — best path to the Root.
- **SW2** → **SW4** and **SW3** → **SW5** each work the same way: SW2/SW3 side = **DP**, SW4/SW5 side = **RP**.
- **SW4** ↔ **SW5** is a direct link between two non-root switches — one side becomes **DP** (forwarding), but since this link creates a **loop** back to the root via two paths, the extra connection between SW4 and SW5 (the bottom link shown) is put into **Blocking** state.
- Result: exactly **one loop-free active path** from every switch back to the Root, with the blocked link kept in reserve as an automatic backup.

7. STP Protocol Types (Variants)

Protocol	Vendor	Instances	Convergence Time	Key Trait
802.1D STP	IEEE	1 tree for ALL VLANs	30–50 s	Original STP; 5 port states
802.1w RSTP	IEEE	1 tree for ALL VLANs	1–6 s	3 states; adds Alternate/Backup port roles
802.1s MSTP	IEEE	Grouped VLANs → instances	1–6 s	Load-balances traffic per instance
PVST+	Cisco	1 tree per VLAN (802.1D-based)	Slow	Per-VLAN root bridge election
RPVST+	Cisco	1 tree per VLAN (802.1w-based)	Fast	Fastest convergence; highest resource usage

Same 3-switch topology, how each protocol treats it:

SW1 (Root) — SW2 — SW3 (identical physical topology for all variants below)

802.1D / 802.1w → ONE tree covers ALL VLANs

802.1s (MSTP) → Instance 1: VLANs 10, 20 | Instance 2: VLAN 30

PVST+ / RPVST+ → A SEPARATE tree per VLAN (VLAN 10, VLAN 20, VLAN 30, ...)

Exam tip: MSTP groups multiple VLANs into a smaller number of instances (efficient), while PVST+/RPVST+ run a completely separate spanning tree per VLAN (more resource-intensive but allows finer per-VLAN load balancing).

8. Port States

8.1 802.1D — 5 States

Blocking → Listening → Learning → Forwarding
↳ (or) Disabled

State	What Happens
Blocking	Receives BPDUs only; does not forward data or learn MAC addresses
Listening	Processes BPDUs to determine port role; still no data forwarding, no MAC learning
Learning	Starts learning MAC addresses into the table; still no data forwarding yet
Forwarding	Fully operational — forwards data and learns MAC addresses
Disabled	Administratively shut down; not participating in STP at all

8.2 802.1w RSTP — 3 States

Discarding → Learning → Forwarding

RSTP consolidates 802.1D's **Blocking + Listening + Disabled** into a single **Discarding** state, making convergence dramatically faster (1–6 seconds vs 30–50 seconds).

802.1D State	Maps To (RSTP)
Blocking	Discarding
Listening	Discarding
Disabled	Discarding
Learning	Learning
Forwarding	Forwarding

0. Full Forms / Abbreviations

Abbreviation	Full Form
AAA	Authentication, Authorization, Accounting

Abbreviation	Full Form
RADIUS	Remote Authentication Dial-In User Service
TACACS+	Terminal Access Controller Access-Control System Plus
ACL	Access Control List
MAC	Media Access Control (address)
UDP	User Datagram Protocol
TCP	Transmission Control Protocol
VPN	Virtual Private Network
ISP	Internet Service Provider

0.1 Topics Covered in These Notes

Topic	Covered In
Standard ACL vs Extended ACL (numbered and named)	\$10
ACL placement — Standard close to destination, Extended close to source	\$10.1
Port Security — violation modes: Protect, Restrict, Shutdown	\$2
MAC Address Learning — Static, Dynamic, and why Sticky MAC is needed (with use case)	\$3
AAA — Authentication, Authorization, Accounting	\$6
TACACS+ vs RADIUS — key differences	\$9

PART A — Port Security

1. What Is Port Security?

Port security limits *which* MAC addresses are allowed to send traffic through a switch **access port** — preventing unauthorized devices (rogue laptops, rogue switches, MAC-spoofing attacks) from connecting to the network.

```
Normal port          : any device can plug in and send traffic
Port-security enabled : only APPROVED MAC address(es) can send traffic
                       everything else = VIOLATION
```

2. Violation Modes

When a port-security violation occurs (an unapproved MAC tries to send traffic, or the max MAC count is exceeded), the switch reacts based on the configured **violation mode**:

Mode	Action	Logs?	Port Stays Up?
Protect	Drops violating frames silently	 No	 Yes
Restrict	Drops frames + logs the event + increments violation counter	 Yes	 Yes
Shutdown	Err-disables the port; stops all traffic on that port (not just the violating frame). Needs to be manually re-enabled by the administrator	 Yes	 No

 **Default on Cisco IOS = Shutdown.** If you don't explicitly specify a violation mode, the port will be **err-disabled** on the very first violation.

Severity ranking (mildest → harshest):

```
Protect < Restrict < Shutdown
(silent)  (logged)  (port goes down completely)
```

Recovering a Shutdown (err-disabled) port:

```
A Shutdown-mode violation requires manual intervention:
Router(config-if)# shutdown
Router(config-if)# no shutdown
(or configure err-disable auto-recovery)
```

3. MAC Address Learning / Types of MAC Address Entries

A switch needs to know **which MAC address is allowed on which port**. There are three ways this gets configured:

3.1 Static MAC

Static = the administrator manually tells the switch exactly which MAC address is allowed on a port.

```
Switch(config-if)# switchport port-security mac-address aabb.cc00.0100
```

- You type the MAC address in yourself.
- It's saved in the running-config right away (and survives reload automatically, since it's a normal config line).
- Best for: a small number of known, fixed devices (e.g., a server that never changes NICs).

3.2 Dynamic MAC

Dynamic = the switch learns it automatically. You don't configure anything.

```
Switch(config-if)# switchport port-security
(no mac-address specified → switch learns whatever connects first, up to the max)
```

- The switch watches incoming frames and learns the source MAC on its own.
- **Downside:** this learned entry is **NOT saved** anywhere — it's lost the moment the switch reloads, meaning it has to re-learn from scratch (and briefly allow whatever connects next) after every reboot.

3.3 Why Is Sticky MAC Needed?

Static and Dynamic each have a real drawback:

Problem	Static MAC	Dynamic MAC
Manual effort	✗ You must type in every MAC address by hand — painful on a switch with dozens/hundreds of ports	✓ No manual effort
Survives reload	✓ Yes	✗ No — has to re-learn after every reboot

Sticky MAC exists to get the best of both: it **learns automatically** like Dynamic MAC (no manual typing), but **writes the learned MAC into the running-config** like Static MAC (so it can persist across reloads) — **provided** you save the config with **write memory**.

```
Static   : ✓ persists      ✗ manual effort
Dynamic  : ✓ no effort     ✗ lost on reload
Sticky   : ✓ no effort     ✓ persists (IF you save config)
          = "best of both worlds"
```

3.4 Sticky MAC — Use Case Scenario

Scenario: You're securing 40 access ports in an office, each connected to a known employee laptop that rarely changes. You want port security enabled, but typing 40 MAC addresses by hand is slow and error-prone, and you also don't want the switch to have to blindly re-learn (and briefly trust) a new device every time it reboots.

```
Step 1: Enable sticky learning on each port:
Switch(config-if)# switchport port-security mac-address sticky

Step 2: Employee laptop connects → switch DYNAMICALLY learns its MAC
        → automatically converts it into a STICKY entry in running-config

Step 3: Admin runs:
Switch# write memory
        → sticky entries are now saved to startup-config
```

Result: After a reboot, the switch already "remembers" every laptop's MAC – no re-learning window, no manual typing of 40 addresses.

Why this matters in practice:

- Saves the admin from manually configuring every single MAC address (unlike pure Static).
- Avoids the reload-vulnerability window where a Dynamic-only port would have to re-learn (and briefly trust) a new device after every switch reboot.
- Ideal for environments where devices are relatively fixed (offices, server rooms) but the sheer number of ports makes manual static configuration impractical.

3.5 Summary Table

Type	How It's Learned	Survives Reload?
Static MAC	Manually configured by the admin	✔ Yes, automatically
Dynamic MAC	Learned automatically from incoming traffic	✗ No — lost on reload
Sticky MAC	Learned automatically (like dynamic) but automatically written into the running-config	✔ Only if write memory (saved to startup-config) is run — otherwise lost on reload

3.6 Sticky MAC — Step-by-Step Flow



Key takeaway: Sticky MAC gives you the *convenience* of dynamic learning with the *persistence* of static configuration — but only if you remember to save the config.

4. Aging Timers

Aging timers control **how long** a dynamically learned secure MAC address stays bound to a port. **By default, secure MACs never age out.**

Type	Behaviour
Absolute	Timer starts the moment the MAC is learned; the MAC is removed when the timer expires — even if the device is still actively sending traffic
Inactivity	Timer resets on every frame received from that MAC; the MAC is only removed after a period of no traffic (idle time)

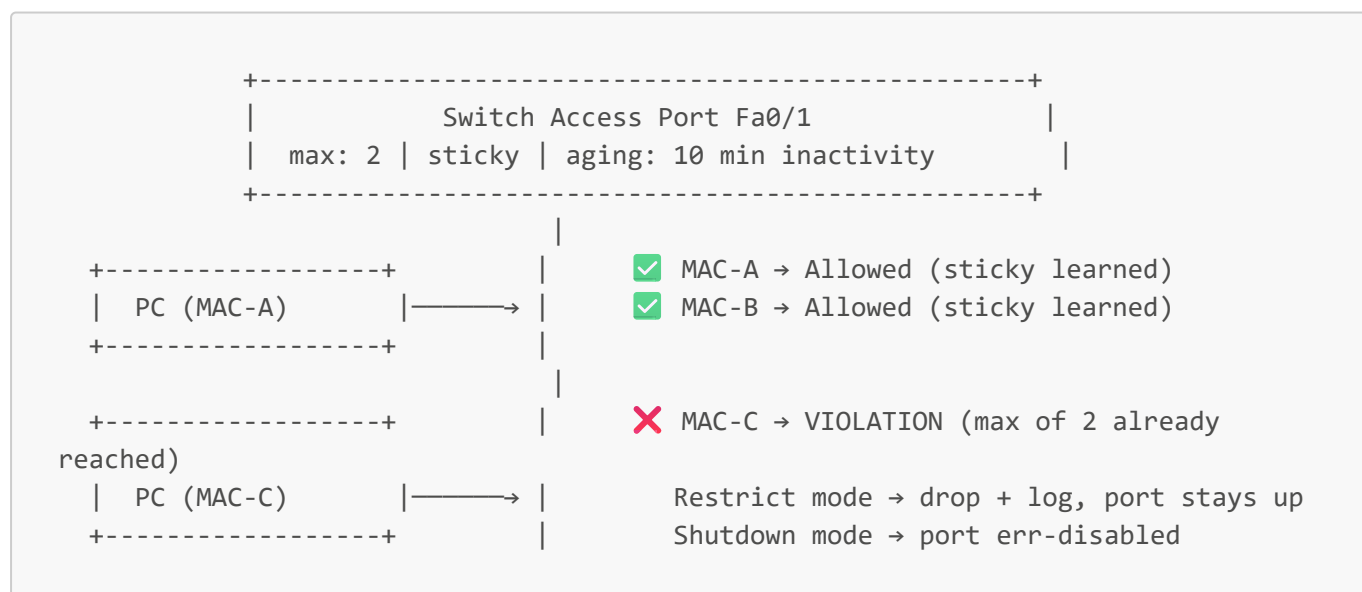
i Static MACs never age out. Only **dynamic** and **sticky** MACs are subject to aging timers.

```

Absolute timer  : |---timer running---| → MAC removed at expiry, REGARDLESS of
activity
Inactivity timer: frame...frame...frame...[idle gap = timer]→ MAC removed only
after idle period

```

5. Port Security — Full Example Diagram



Reading this example:

- Port Fa0/1 is configured for a **max of 2** MAC addresses, using **sticky** learning, with a **10-minute inactivity** aging timer.
- MAC-A and MAC-B are learned first and allowed (sticky-learned, within the max of 2).
- MAC-C attempts to connect — but the port has already reached its max of 2 allowed MACs → **violation**.
- What happens next depends on the configured **violation mode** (§2): silently dropped (Protect), dropped+logged (Restrict), or the whole port goes down (Shutdown).

PART B — AAA Framework (Authentication, Authorization, Accounting)

6. What Is AAA?

Function	Question Answered	Examples
Authentication	"Who are you?"	Username/password, certificates, tokens
Authorization	"What can you do?"	Exec-privilege level, which commands are allowed
Accounting	"What did you do?"	Session logs, command history, session duration

AAA Flow:

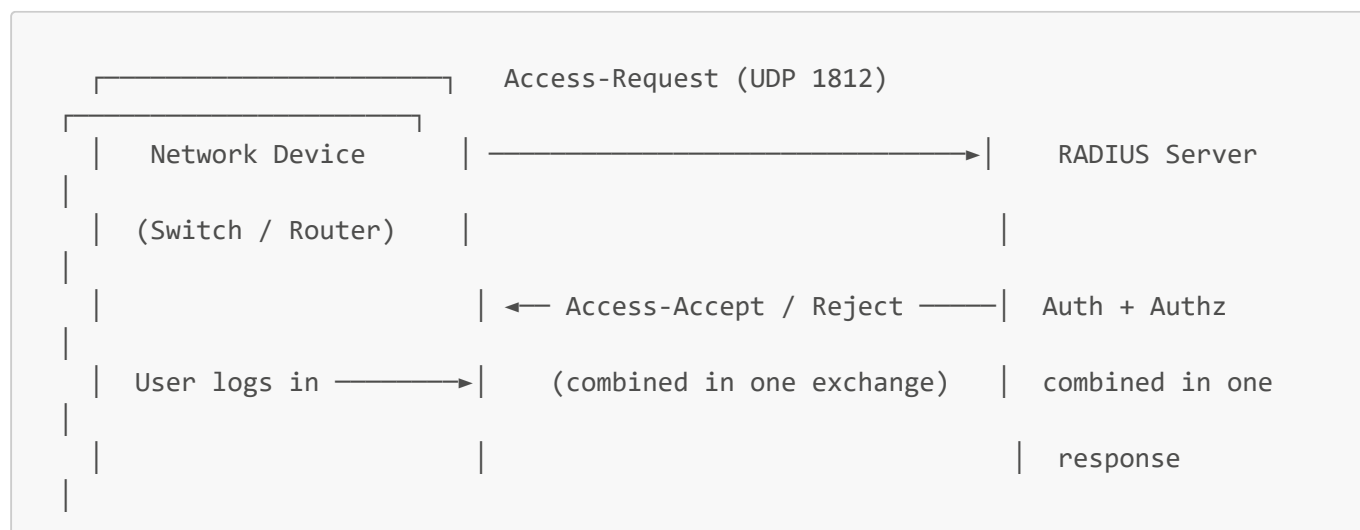
1. Authentication → prove identity (login)
2. Authorization → determine permitted actions
3. Accounting → record what was actually done

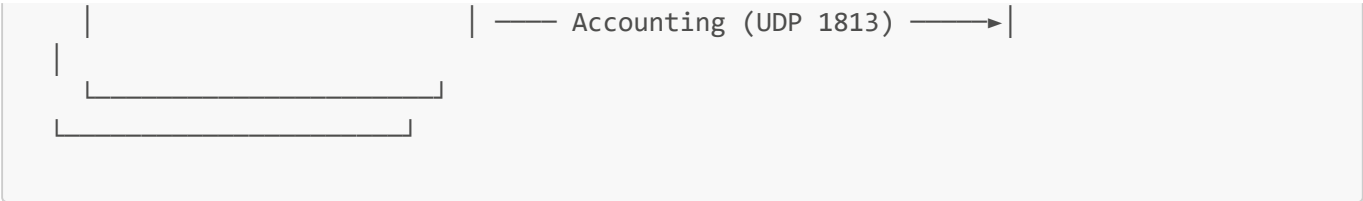
AAA is implemented using one of two main protocols: **RADIUS** or **TACACS+**.

7. RADIUS

RADIUS = Remote Authentication Dial-In User Service. An **open standard**, client-server AAA protocol.

- **Transport:** UDP
- **Ports:** **1812** (Authentication) and **1813** (Accounting)
- **Key trait:** Authentication + Authorization are **combined** into a single exchange (not separated)



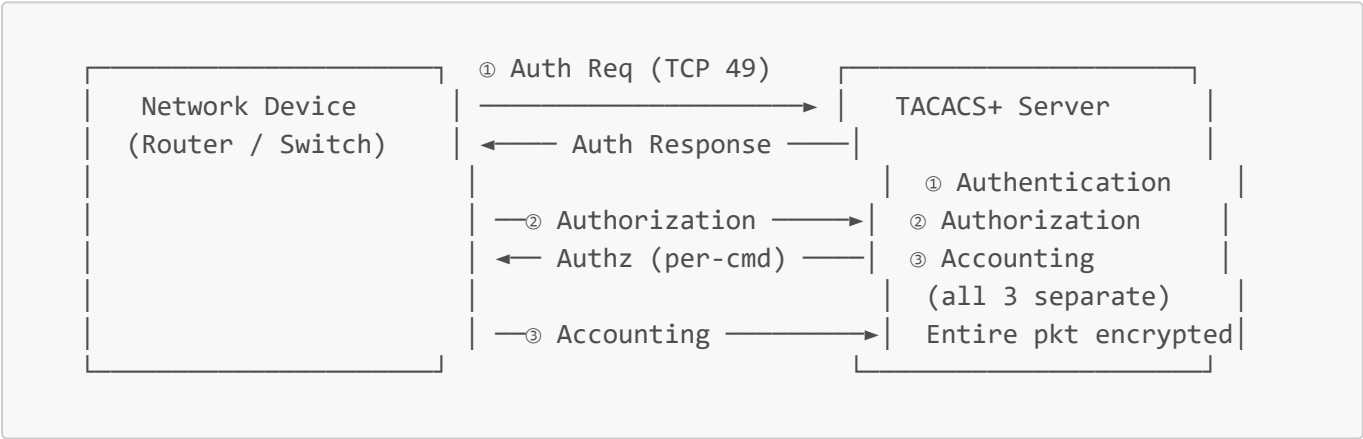


Common use cases: Wireless authentication, VPN access, ISP subscriber authentication.

8. TACACS+

TACACS+ = Terminal Access Controller Access-Control System Plus. A **Cisco-enhanced/proprietary** AAA protocol.

- **Transport:** TCP
- **Port:** 49
- **Key trait:** Authentication, Authorization, and Accounting are **fully separated** into three independent exchanges — enabling **per-command authorization**



Common use case: Device administration (Cisco environments) — e.g., controlling exactly which CLI commands a network admin is allowed to run.

9. RADIUS vs TACACS+

Feature	RADIUS	TACACS+
Standard	Open (RFC 2865)	Cisco proprietary
Transport	UDP	TCP
Ports	1812 / 1813	49
Separates A/A/A?	✗ No (Auth + Authz combined)	✓ Yes (all three fully separate)
Per-command authorization	✗ Limited	✓ Full support
Encrypts packet body?	✗ Password only	✓ Entire packet
Common use	Wireless, VPN, ISP	Device administration (Cisco)

Exam one-liners:

- RADIUS = UDP, open standard, combines Auth+Authz, encrypts only the password.
- TACACS+ = TCP, Cisco proprietary, fully separates AAA, encrypts the entire packet, supports per-command control.
- Choose **TACACS+** for fine-grained device-administration control; choose **RADIUS** for general network access (Wi-Fi, VPN, ISP).

PART C — Access Control Lists (ACLs)

10. ACL Types

Type	Filters By	Number Range	Placement
Standard ACL	Source IP only	1–99, 1300–1999	Close to the destination
Extended ACL	Source/Destination IP, protocol, port	100–199, 2000–2699	Close to the source
Named ACL	Same filtering as Standard/Extended, but identified by name instead of number	N/A	Same placement rules as its Standard/Extended equivalent

10.1 ACL Configuration Commands

A. Standard ACL (Numbered) — filters by source IP only

Step 1 – Create the ACL:

```
Router(config)# access-list 10 permit 192.168.1.0 0.0.0.255
Router(config)# access-list 10 deny any
```

Step 2 – Apply it to an interface (close to the DESTINATION):

```
Router(config)# interface g0/1
Router(config-if)# ip access-group 10 out
```

Part	Meaning
<code>access-list 10</code>	ACL number 10 → falls in the Standard range (1–99)
<code>permit 192.168.1.0 0.0.0.255</code>	Allow traffic sourced from the 192.168.1.0/24 network (wildcard mask — see earlier notes)
<code>ip access-group 10 out</code>	Apply ACL 10 to traffic leaving this interface

B. Extended ACL (Numbered) — filters by source/dest IP, protocol, port

Step 1 – Create the ACL:

```
Router(config)# access-list 110 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80
Router(config)# access-list 110 deny ip any any
```

Step 2 – Apply it to an interface (close to the SOURCE):

```
Router(config)# interface g0/0
Router(config-if)# ip access-group 110 in
```

Part	Meaning
<code>access-list 110</code>	ACL number 110 → falls in the Extended range (100–199)
<code>permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.5 eq 80</code>	Allow TCP traffic from 192.168.1.0/24 to host 10.0.0.5, destination port 80 only
<code>ip access-group 110 in</code>	Apply ACL 110 to traffic entering this interface

C. Named ACL — same logic, human-readable name instead of a number

Standard Named ACL:

```
Router(config)# ip access-list standard BLOCK-SALES
Router(config-std-nacl)# deny 192.168.10.0 0.0.0.255
Router(config-std-nacl)# permit any
```

Extended Named ACL:

```
Router(config)# ip access-list extended ALLOW-WEB
Router(config-ext-nacl)# permit tcp any host 10.0.0.5 eq 443
Router(config-ext-nacl)# deny ip any any
```

Apply (same as numbered):

```
Router(config-if)# ip access-group BLOCK-SALES in
Router(config-if)# ip access-group ALLOW-WEB in
```

Exam one-liners:

- Standard ACL syntax: `access-list <1-99> {permit|deny} <source-ip> <wildcard-mask>`
- Extended ACL syntax: `access-list <100-199> {permit|deny} <protocol> <source> <destination> [eq port]`
- `ip access-group <ACL> {in|out}` binds the ACL to an interface, in a specific direction.
- Named ACLs use `ip access-list {standard|extended} <name>` instead of a number, then enter a sub-mode to add `permit/deny` lines.

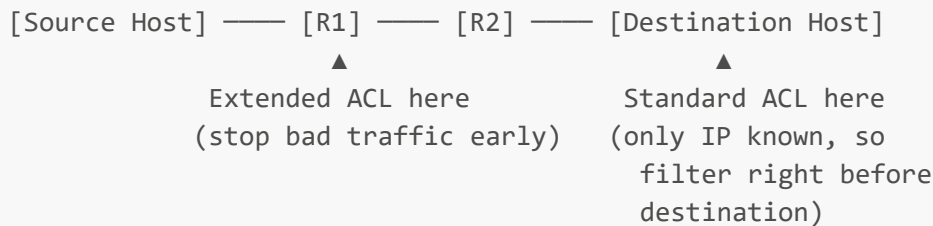
10.2 Why Placement Matters

Standard ACL (source IP only) → place NEAR the DESTINATION

Reason: it can only match source IP, so placing it near the source would block traffic from reaching OTHER destinations it shouldn't affect.

Extended ACL (full 5-tuple match) → place NEAR the SOURCE

Reason: it can match precisely what it needs to block, so it's safe (and more efficient) to stop unwanted traffic as early as possible.



10.3 Implicit Deny

⚠ Implicit Deny: Every ACL ends with an **invisible deny any** statement. If a packet doesn't match **any** configured rule, it is **automatically dropped**.

ACL processing order:

Rule 1 → match? → apply action, STOP

Rule 2 → match? → apply action, STOP

...

Rule N → match? → apply action, STOP

[implicit deny any] → no match found anywhere → DROP

Exam tip: If you want to allow general traffic through an ACL, you must explicitly add a **permit** statement — otherwise the implicit deny silently blocks everything not explicitly permitted.

0. Full Forms / Abbreviations

Abbreviation	Full Form
VC / VCI	Virtual Circuit / Virtual Circuit Identifier
FCS	Frame Check Sequence
CRC	Cyclic Redundancy Check
CSMA/CD	Carrier Sense Multiple Access with Collision Detection
MAC	Media Access Control (address)
NIC	Network Interface Card

PART A — Network Switching Types

These describe the fundamental ways data can travel through a network of intermediate nodes. This is general networking theory — separate from how an individual Ethernet switch behaves internally, which is covered in Part B.

1. Circuit Switching

A **dedicated physical path** is set up end-to-end **before** any data is sent. The whole session then uses that **same fixed path** — exactly like a traditional telephone call.

Phases:

1. Setup — Call request and connection setup
2. Data Transfer — Data flows continuously over the fixed path
3. Teardown — Connection is released once transfer completes



Dedicated path reserved for the ENTIRE call.
No other traffic can use it — and it sits idle
(wasting capacity) whenever no data is flowing.

✓ Pros

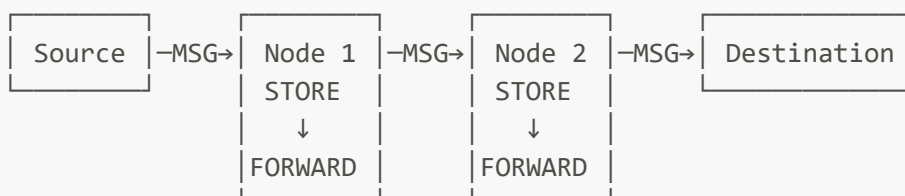
Low delay during transfer — the path is always ready and reserved

✗ Cons

Wastes bandwidth whenever the path is idle but still reserved

2. Message Switching

The **entire message** is treated as one unit. Each intermediate node **stores the whole message**, then forwards it onward once ready — a "**store-and-forward**" approach applied at the message level.



The entire message must be stored at EACH node before it can be forwarded.
This causes high delay — especially for large messages.

✓ Pros

Efficient for bursty traffic (no dedicated path needed)

✗ Cons

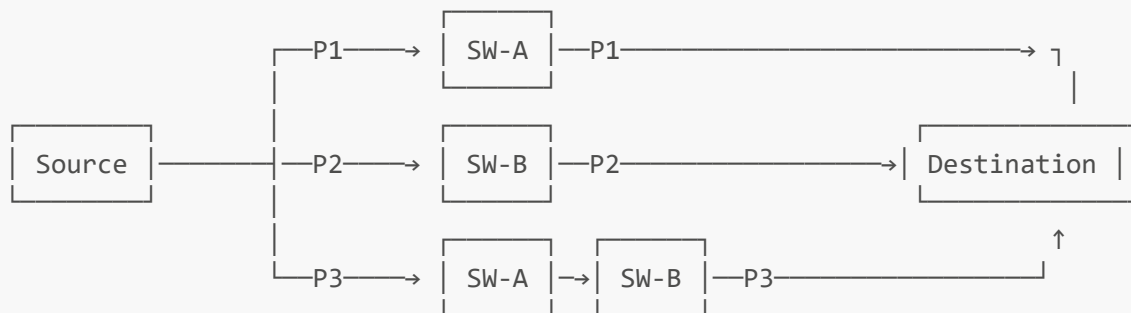
High delay from storing full messages at every hop; rarely used today

3. Packet Switching

The message is broken into smaller **packets** that travel independently or semi-independently through the network. There are two sub-types.

3.1 Datagram Switching (Connectionless)

Each packet is **routed independently** — there's no fixed path. Different packets from the same message may take **different routes**, and they're only reassembled at the receiver. Every packet must carry the **full source and destination address**, since each one is routed on its own.



- Each packet carries the FULL source & destination address.
- Packets can arrive OUT OF ORDER (e.g. P3 might arrive before P1).
- Reassembly happens only at the Destination.

✓ Pros

Efficient, and resilient to link failures (packets simply reroute around a broken link)

✗ Cons

Delay is unpredictable; out-of-order arrival is possible

3.2 Virtual Circuit Switching (Connection-Oriented)

A **logical path** (the virtual circuit) is set up **before** data transfer begins. Every packet then follows that **same pre-decided route**, identified by a short **Virtual Circuit Identifier (VCI)** instead of carrying a full address in every packet.

Phases:

1. VC Establishment — A setup packet reserves the logical path
2. Data Transfer — All packets follow the established VC
3. VC Teardown — The logical connection is released

Phase 1 - VC Establishment (logical path reserved):



Virtual Circuit VC1 (pre-established)

- Only a VCI label travels in each packet header
- No per-packet routing decision needed

Phase 2 - Data Transfer (all packets follow VC1, arrive in order):

[VC1, P1] → → → → → In-order delivery ✓

[VC1, P2] → → → → →

[VC1, P3] → → → → →

Phase 3 - VC Teardown: connection released, resources freed

✓ Pros

Low per-packet delay; delivery is guaranteed in order

✗ Cons

Less resilient to link failure (the whole VC breaks); has setup overhead

3.3 Quick Comparison — All Switching Types

Type	Path	Addressing	Connection-Oriented?	Arrival Order
Circuit Switching	Fixed, dedicated	None needed (path is pre-set)	Yes (end-to-end)	In order
Message Switching	Per message	Full address in the message	No	In order
Datagram (Packet)	Different per packet	Full address in every packet	No	May be out of order
Virtual Circuit	Same logical path	VCI only	Yes	In order

One-liners:

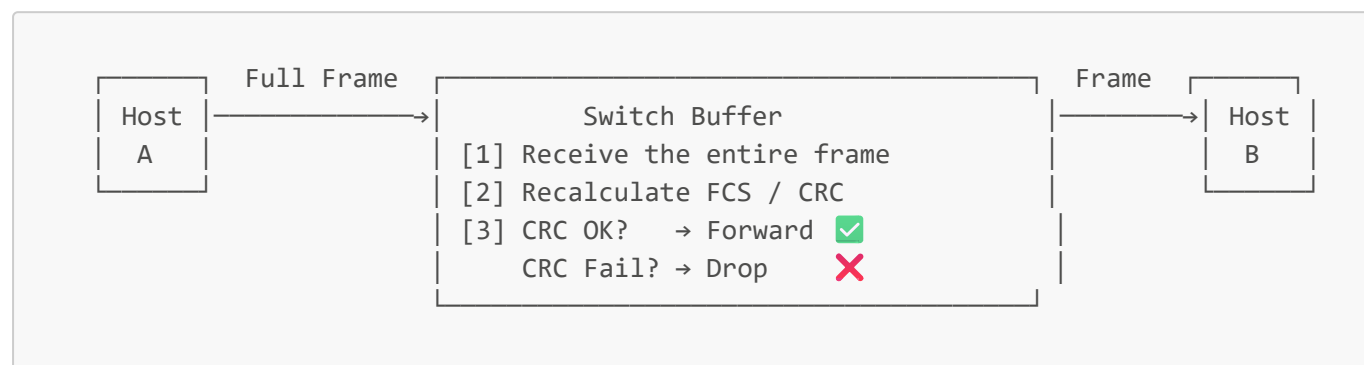
- **Circuit switching** = dedicated path, wasteful if idle, low delay once set up — think phone calls.
- **Message switching** = store-and-forward at the whole-message level, high delay, largely obsolete.
- **Datagram switching** = no fixed path, out-of-order possible, resilient — classic IP routing behavior.
- **Virtual Circuit switching** = fixed logical path + VCI, in-order, low per-packet overhead — think MPLS, Frame Relay, ATM.

PART B — Ethernet Forwarding Methods

Part A described how data moves *across a network*. Part B zooms into a single Ethernet switch and asks: once a frame starts arriving on a port, **when does the switch begin forwarding it out the other side?**

4. Store-and-Forward Switching

The switch receives the **entire frame** into its buffer **before** making any forwarding decision. It checks the **FCS (CRC)** for errors and **drops invalid frames**.



✓ Pros

Most reliable — bad frames never leave the switch

✗ Cons

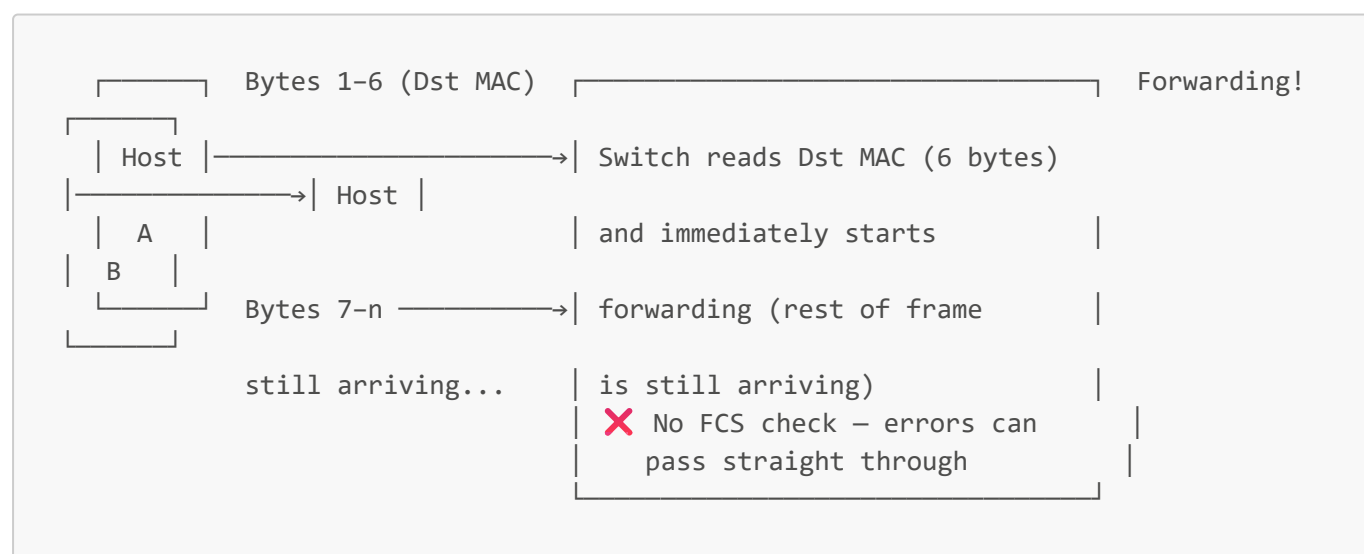
Highest latency of all methods (has to wait for the whole frame)

5. Cut-Through Switching

Forwarding **starts before the whole frame has arrived**. There are two variants.

5.1 Fast-Forward Switching

Starts forwarding **the instant the destination MAC address is read** — just the **first 6 bytes** of the frame. It does **not** check the FCS at all.



✓ Pros

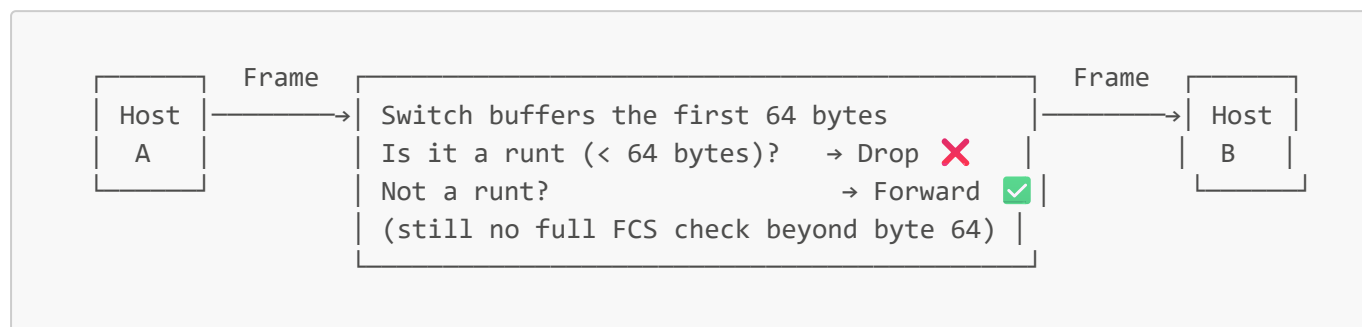
Lowest latency of all switching methods

✗ Cons

Can forward corrupted frames — no error checking at all

5.2 Fragment-Free Switching

An improved cut-through variant: the switch buffers the **first 64 bytes** before forwarding. Ethernet collisions produce fragments **shorter than 64 bytes** (called "runt"), so this catches and filters those out. It still does **not** perform a full FCS check.



✓ Pros

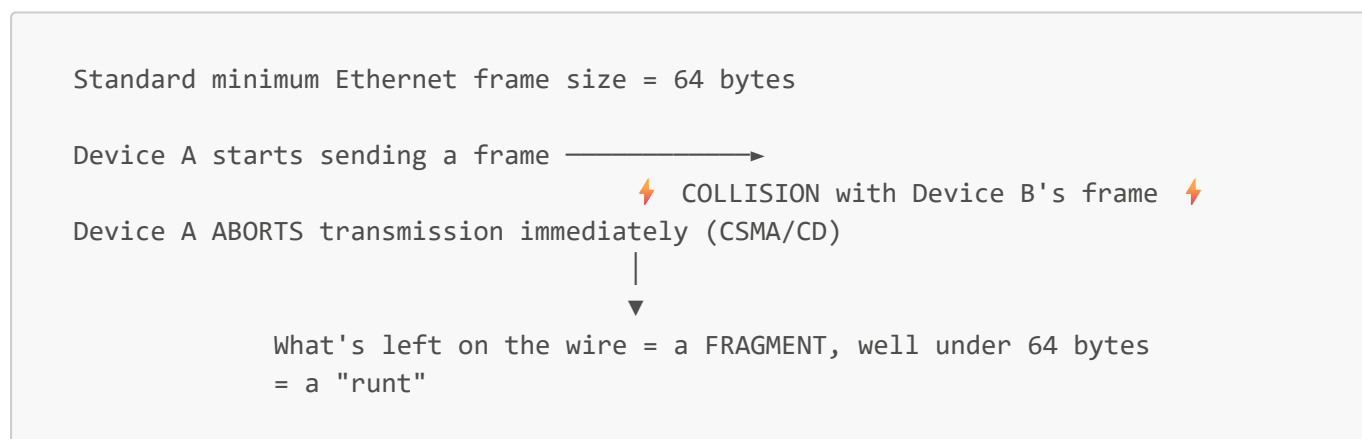
✗ Cons

Filters out collision fragments (runt) Doesn't catch bit errors that occur beyond byte 64

5.3 What Exactly Is a "Runt" (Collision Fragment)?

A **runt** is a very small, incomplete frame left over on the wire after a **collision**.

Why it happens: On a **shared, half-duplex** segment (Section 8), two devices can start transmitting at nearly the same instant. When their signals meet on the wire, they collide and corrupt each other. As soon as a device's CSMA/CD logic (Section 7) detects this, it **immediately stops transmitting** that frame. What's left behind on the wire is a **partial, truncated frame** — not a real, usable one.



What makes a runt a runt:

- **Size** — under the Ethernet minimum of **64 bytes** (often as small as ~40–60 bytes, depending on exactly when the collision hit).
- **Cause** — an aborted transmission from a collision, not a deliberately sent complete frame.
- **Validity** — meaningless, corrupted data; it's only part of a frame, so there's nothing usable in it.
- **Risk if forwarded** — a switch that doesn't check for runts wastes downstream bandwidth passing along garbage data.

Why this is exactly why Fragment-Free checks 64 bytes: a genuine collision fragment can only ever be shorter than 64 bytes — that's simply how fast CSMA/CD detects and aborts a transmission. So buffering the

first 64 bytes before forwarding guarantees that anything shorter is, by definition, a runt — never a legitimate frame — and it's safe to drop.

- **Fast-Forward** (Section 5.1) skips this check entirely — since it starts forwarding after only 6 bytes, it can end up pushing runts onto the rest of the network.
- **Store-and-Forward** (Section 4) filters runts too, just as a side effect of waiting for and validating the complete frame anyway.

One-liners:

- Runt = collision fragment = a frame **shorter than the 64-byte Ethernet minimum**, caused by two devices transmitting at once and one aborting mid-frame.
- Runts only happen on **half-duplex/shared media** — they can't occur on collision-free full-duplex links.
- Fragment-Free switching buffers exactly 64 bytes because that's the guaranteed minimum size of any *legitimate* frame — which makes the runt check reliable at that exact cutoff.

5.4 Quick Comparison — Ethernet Forwarding Methods

Method	Buffers	Error Check	Latency	Best For
Store-and-Forward	Entire frame	Full FCS/CRC	Highest	Reliability
Fast-Forward	First 6 bytes (Dst MAC)	None	Lowest	Raw speed
Fragment-Free	First 64 bytes	Runt/collision check only	Medium–Low	Balance of speed and basic error filtering

One-liners:

- **Store-and-Forward** = safest, slowest — waits for the whole frame and runs a full CRC check.
- **Fast-Forward** = fastest, riskiest — forwards right after reading the destination MAC.
- **Fragment-Free** = middle ground — waits for 64 bytes to filter out runts, but skips the full CRC check.
- Most modern switches default to **Store-and-Forward**; cut-through variants show up mainly in very latency-sensitive environments (e.g. some data-center or high-frequency-trading switching).

PART C — Layer 2 Switching Fundamentals

Part B explained *when* a switch starts forwarding a frame. Part C covers the bigger picture of day-to-day switch operation: how it learns where devices are, what it does when it doesn't know, and how the underlying link (half- vs full-duplex) affects all of it.

6. The MAC Address Table, Flooding & Forwarding

A Layer 2 switch builds and uses a **MAC address table** (also called a **CAM table** — Content Addressable Memory table) to decide where to send each frame.

6.1 How the Switch Learns (Building the MAC Table)

Step 1: A frame arrives on a port.
 Step 2: The switch reads the frame's SOURCE MAC address.
 Step 3: It records {Source MAC ↔ Incoming Port} in its MAC address table.
 Step 4: That entry is then used for FUTURE frames destined to that MAC.

Host A (MAC-A) —frame→ Switch Port Fa0/1
 ↓
 MAC table learns:
 MAC-A ↔ Fa0/1

6.2 The Forwarding Decision

When a frame arrives, the switch checks its MAC table for the DESTINATION MAC:

Is the destination MAC already in the table?
 └ YES → Forward out that ONE specific port (efficient, unicast)
 └ NO → Flood out ALL ports except the one the frame arrived on

6.3 Flooding

Flooding happens when the switch doesn't yet know which port leads to the destination MAC — so it sends a copy of the frame out **every port** except the one it came in on. The hope is that the real destination is out there somewhere and will eventually reply, letting the switch learn its actual port.

Frame with an UNKNOWN destination MAC:

Host A —frame→ Switch —┬→ Port 2 (flooded)
 └→ Port 3 (flooded)
 └→ Port 4 (flooded)
 (NOT sent back out Port 1 – the port it arrived on)

Flooding also happens for:

- **Broadcast frames** (destination **FFFF.FFFF.FFFF**) — always flooded to every port.
- **Multicast frames** — flooded unless multicast-aware filtering (e.g. IGMP snooping) is configured.
- **Unknown unicast** — as above, until the switch learns that MAC's real port.

6.4 A Full Example: Learn → Flood → Forward

T1: Host A sends a frame to Host B (switch doesn't know Host B's port yet)
 → Switch learns Host A's MAC ↔ Port 1
 → Switch FLOODS the frame out every other port (Host B's real port is one of

them)

T2: Host B replies to Host A

→ Switch learns Host B's MAC ↔ Port 3

→ Switch already knows Host A is on Port 1 → FORWARDS directly, no flooding needed

T3 onward: Switch now knows BOTH MACs

→ All further traffic between A and B is forwarded directly, port to port

One-liners:

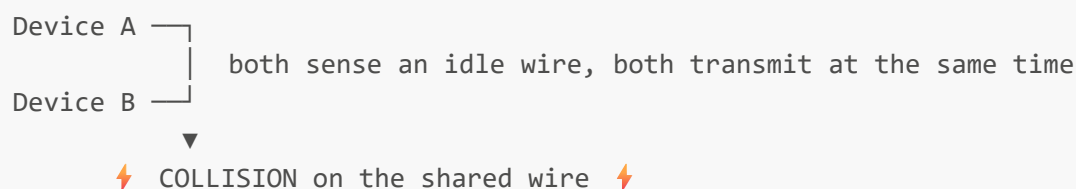
- The MAC table maps **MAC address** ↔ **switch port**, learned from the **source MAC** of incoming frames.
- **Known destination MAC** → forward out that one port only.
- **Unknown destination MAC, broadcast, or unhandled multicast** → flood out every port except the source port.
- Flooding is a **fallback**, not the switch's normal behavior — it stops as soon as the destination's real port is learned.

7. CSMA/CD — Carrier Sense Multiple Access with Collision Detection

CSMA/CD is the access method traditionally used by Ethernet on **shared, half-duplex media** (old hub-based networks or coax Ethernet) to decide when a device may transmit, and to recover when two devices transmit at once (a **collision**).

7.1 How CSMA/CD Works, Step by Step

1. Carrier Sense : Before transmitting, the device LISTENS to the wire.
Idle? → proceed. Busy? → wait.
2. Multiple Access : Many devices share the SAME medium and can all attempt to transmit – there's no central controller granting turns.
3. Collision Detect: If two devices transmit at the SAME time, their signals collide and corrupt on the wire. Both devices detect this.
4. Jam Signal : The colliding devices send a jam signal so that EVERY device on the segment knows a collision occurred.
5. Backoff & Retry : Each device waits a RANDOM amount of time, then retries – reducing the odds of an immediate repeat collision.





7.2 When Is CSMA/CD Actually Used?

- It matters on **half-duplex** links, where collisions are physically possible (Section 8).
- It's **not needed on full-duplex** links (standard on modern switched Ethernet) — since sending and receiving use separate paths, **collisions can't happen**, so CSMA/CD is effectively irrelevant there.

One-liners:

- CSMA/CD = Carrier Sense (listen first) + Multiple Access (shared medium) + Collision Detection (detect and recover from simultaneous transmissions).
- Collision-handling sequence: **sense** → **transmit** → **detect collision** → **jam signal** → **random backoff** → **retry**.
- Mostly a **legacy** concept today — largely irrelevant on modern full-duplex switched networks, but still an important exam topic.

8. Half-Duplex vs Full-Duplex

Feature	Half-Duplex	Full-Duplex
Data flow	One direction at a time (send <i>or</i> receive)	Both directions at once (send <i>and</i> receive)
Collisions possible?	✓ Yes	✗ No
CSMA/CD needed?	✓ Yes	✗ No
Typical use today	Legacy hubs, some wireless links	Standard for modern switched Ethernet (switch-to-host, switch-to-switch)
Throughput	Lower — shared channel, contention overhead	Higher — dedicated send/receive paths, no contention

Half-Duplex:

A —send→
A ←recv— (not at the same

Full-Duplex:

A —send→
A ←recv— (send & recv happen

time)

SIMULTANEOUSLY, on
separate wire pairs)

Why modern networks run full-duplex:

- A switch (unlike an old hub) gives each device its **own dedicated collision domain** per port.
- With a direct switch-to-host link and no shared medium, a collision **can't physically happen** — so both ends can safely send and receive at the same time.

One-liners:

- **Half-duplex** = one direction at a time, collisions possible, needs CSMA/CD.
- **Full-duplex** = both directions at once, no collisions, CSMA/CD not needed.
- Modern switched links (switch ↔ host, switch ↔ switch) run **full-duplex** by default.
- A **duplex mismatch** (one side set to half, the other to full) is a classic real-world cause of poor performance and errors on a link.

0. Full Forms / Abbreviations

Abbreviation	Full Form
POST	Power-On Self-Test
IOS	Internetwork Operating System (Cisco)
ROM	Read-Only Memory
RAM	Random-Access Memory
NVRAM	Non-Volatile Random-Access Memory
Flash	Flash Memory
SSH	Secure Shell
Telnet	Teletype Network
CLI	Command-Line Interface
TFTP	Trivial File Transfer Protocol
ROMmon	ROM Monitor (also written RXBOOT)
VTY	Virtual Teletype (virtual terminal line, used for remote CLI sessions)
AUX	Auxiliary (port)

PART A — What Is Cisco IOS?

Cisco IOS is the operating system that runs on Cisco routers and switches. It is responsible for:

- **Routing** — moving packets between networks
- **Switching** — forwarding frames within a network

- **Network device management** — CLI, configuration, monitoring
- **Security** — passwords, ACLs, AAA, encryption features (e.g., SSH)

PART B — Router Hardware Components (The "Big 5")

Component	Role
CPU	Executes IOS instructions and processes
RAM	Stores the running-config, routing tables, ARP cache, buffers, and the currently loaded/running IOS
ROM	Stores POST routines, the Bootstrap program, and a bare-bones mini-IOS (ROMmon)
NVRAM	Stores the startup-config
Flash	Stores the full IOS image file (the OS itself)

Memory Cheat Sheet

Memory Type	Stores	Volatile?	Analogy
ROM	Bootstrap program, POST routines, mini-IOS (ROMmon)	✗ No — permanent	The router's built-in "BIOS"
Flash	The full IOS image	✗ No — persists across reboots	The router's "hard drive"
NVRAM	The startup-config	✗ No — persists across reboots	Saved settings file
RAM	The running-config + currently running IOS + routing tables/ARP cache/buffers	✓ Yes — wiped on reboot/power loss	The router's "working memory"

Memory mnemonic: "Real Networks Feel Real" → **R**AM (Running-config) → **N**VRAM (Startup-config) → **F**lash (Full IOS) → **R**OM (ROMmon/Bootstrap).

Key Distinctions

Concept	Lives In	Notes
<code>show running-config</code>	RAM	The currently active config — lost on reboot unless saved
<code>show startup-config</code>	NVRAM	The config that will load at the next boot
<code>copy running-config startup-config</code> (or <code>write memory</code>)	RAM → NVRAM	Saves the active config so it survives a reboot

Concept	Lives In	Notes
IOS image file (.bin)	Flash	The actual OS software; Flash can hold multiple IOS versions
ROMmon / mini-IOS	ROM	Emergency recovery mode if the real IOS can't load

PART C — Cisco IOS CLI Modes

Mode	Purpose	Prompt	How to Enter
A. User EXEC	Limited access — basic monitoring commands only (e.g., <code>ping</code> , <code>show version</code>)	<code>Router></code>	Default mode on login
B. Privileged EXEC	Full administrative access — debugging, copying/saving config, entering config mode	<code>Router#</code>	<code>Router> enable</code>
C. Global Configuration	Configure system-wide settings (hostname, passwords, routing, etc.)	<code>Router(config)#</code>	<code>Router# configure terminal</code>
D. Interface Configuration	Configure a specific interface (IP address, description, etc.)	<code>Router(config-if)#</code>	<code>Router(config)# interface <name></code>
E. Line Configuration	Configure console / VTY / AUX lines (passwords, login method, transport)	<code>Router(config-line)#</code>	<code>Router(config)# line vty 0 4 (etc.)</code>

Common User EXEC Commands

```
Router> show ?
Router> show history
Router> show version
Router> ping 192.168.1.1
Router> traceroute 8.8.8.8
Router> telnet 192.168.1.10
Router> logout
```

Privileged EXEC — Common Uses

Once in `Router#`, you can:

- View detailed information (`show running-config`, `show startup-config`, `show version`)
- Copy/save configuration
- Run `debug` commands
- Enter global configuration mode

```
Router# show running-config
```

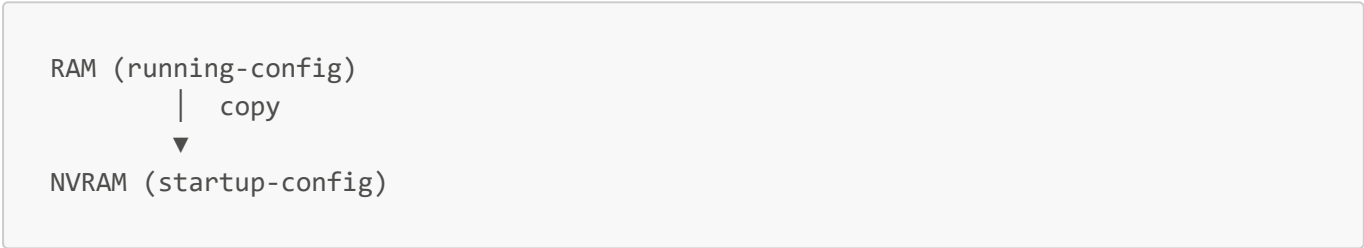
Shows the **active** configuration in RAM. Includes: interfaces, hostname, passwords, routing configuration, VTY line settings, and enabled services.

```
Router# show startup-config
```

Shows the configuration **saved in NVRAM** that will be used on the next reboot.

```
Router# copy running-config startup-config
```

Saves the current (active) configuration to NVRAM so it survives a reboot:



Types of Line Passwords

- **Console password** — protects local console-port access (`line console 0`)
- **Telnet / SSH password** — protects remote CLI access (`line vty 0 4`)

PART D — Router Boot Sequence

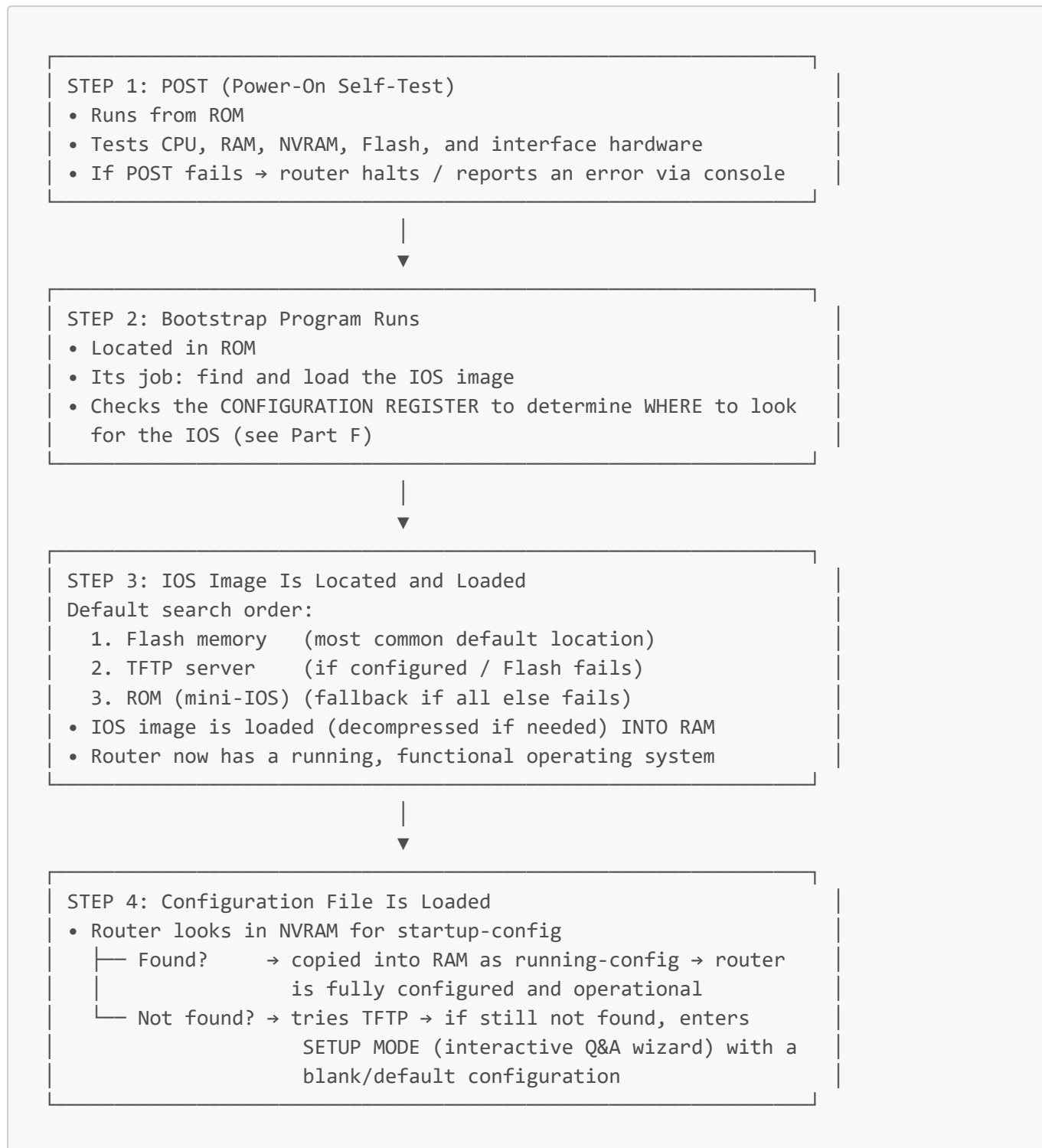
1. Boot Sequence Overview

```
POST → Bootstrap → Locate & Load IOS → Load Configuration
```

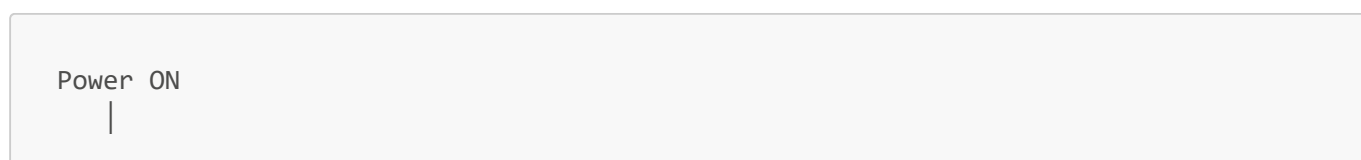
Stage	What Happens	Stored In
1. POST (Power-On Self-Test)	Hardware diagnostic check — CPU, RAM, interfaces, and other components are tested for basic functionality	Executed from ROM
2. Bootstrap	A small program that locates and loads the IOS image	Stored in ROM
3. IOS Load	The bootstrap program loads the Cisco IOS image into RAM, following the order set by the configuration register	Loaded from Flash (default) into RAM

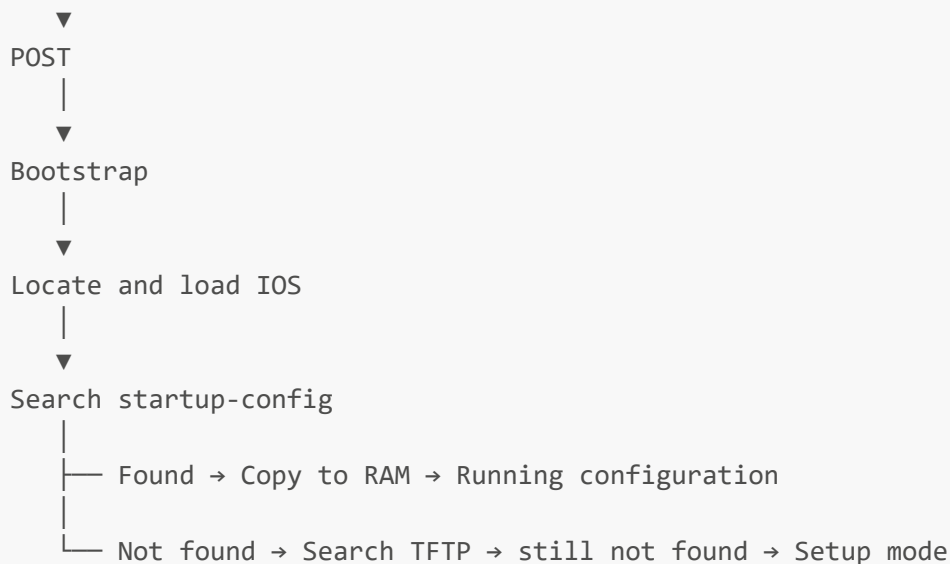
Stage	What Happens	Stored In
4. Configuration	The router loads its saved configuration (startup-config) into running memory to become operational	Loaded from NVRAM into RAM

1.1 Detailed Step-by-Step Flow



Simplified Flow (as commonly drawn)





Exam One-Liners

- Order to remember: **POST** → **Bootstrap** → **IOS** → **Config** ("Please **B**ring **I**ce **C**ream").
- POST and Bootstrap both run from **ROM**.
- IOS is normally loaded from **Flash**; startup-config is loaded from **NVRAM**.
- If startup-config is missing, the router tries **TFTP**, and if that also fails, it enters **Setup Mode**.

PART E — Telnet vs SSH (Remote Access Security)

Both protocols allow **remote CLI access** to a router/switch through **VTY lines**, but they differ fundamentally in security.

By default a router has 5 VTY lines (`line vty 0 4` = VTY 0 through VTY 4), meaning up to 5 simultaneous remote CLI sessions.

Feature	Telnet	SSH
Encryption	❌ None — sends everything, including passwords, in plain text	✅ Full encryption of the entire session
Port	TCP 23	TCP 22
Authentication security	Weak — credentials easily captured via packet sniffing	Strong — supports password AND public-key authentication
Data integrity checking	❌ No	✅ Yes (cryptographic integrity checks)
Vulnerable to eavesdropping?	✅ Very — anyone on the path can read the traffic	❌ No — traffic is encrypted end-to-end
Recommended today?	❌ No — legacy/insecure, mainly seen in labs/older environments	✅ Yes — industry standard for remote device management

Why Telnet Is Insecure — Visualized

Telnet Session (PLAIN TEXT):

Admin PC — "Username: admin" —> Router
Admin PC — "Password: Cisco123" —> Router



Anyone capturing packets on this path
(e.g., via Wireshark) can read the
password DIRECTLY – no decryption needed.

Example command: `telnet 192.168.1.1`

Why SSH Is Secure — Visualized

SSH Session (ENCRYPTED):

Admin PC — [encrypted blob: xK9\$#mP2...] —> Router



An eavesdropper only sees scrambled
ciphertext – username/password/commands
are unreadable without the correct key.

Example command: `ssh admin@192.168.1.1`

Basic SSH Configuration (Cisco IOS)

Step 1 – Set hostname and domain name (required for RSA key generation):

```
Router(config)# hostname R1  
R1(config)# ip domain-name mylab.com
```

Step 2 – Generate the RSA key pair:

```
R1(config)# crypto key generate rsa  
(choose a modulus size, e.g. 1024 or 2048 bits)
```

Step 3 – Create a local user account:

```
R1(config)# username admin secret StrongPass123
```

Step 4 – Configure the VTY lines to use SSH + local login:

```
R1(config)# line vty 0 4  
R1(config-line)# transport input ssh  
R1(config-line)# login local
```

`transport input ssh` **disables Telnet** on the VTY lines (only SSH is accepted). Using `transport input telnet ssh` would allow both — not recommended for security.

- Telnet = **unencrypted**, TCP port 23 — never use on production networks.
- SSH = **encrypted**, TCP port 22 — the modern standard for remote device administration.
- SSH requires a **hostname + domain name + RSA key pair** before it can be enabled.
- Best practice: **transport input ssh** on VTY lines to **disable Telnet entirely**.
- Default VTY lines: **0 to 4** (5 lines total).

PART F — Ways to Access the Cisco IOS CLI

There are four common ways an administrator can reach the CLI of a router:

1. Console — Local, Out-of-Band Access

PC/Laptop — Console cable — Router

Direct physical connection to the router's console port, accessed via a terminal program.

Used when:

- Router is being configured for the first time (no network config exists yet)
- Network access is unavailable
- Password recovery is needed

2. SSH — Remote, Encrypted Access (Preferred)

Admin PC — SSH → Router

Example: `ssh admin@192.168.1.1`

Used when: administering the router remotely and secure access is required. SSH encrypts the entire communication (see Part E).

3. Telnet — Remote, Unencrypted Access

Admin PC — Telnet → Router

Example: `telnet 192.168.1.1`

Used when: remote CLI access is needed — typically only in labs or older environments. ⚠️ Not recommended for production since credentials and data are sent in plain text.

4. AUX Port — Out-of-Band Remote/Modem Access

```
Remote Admin — Modem — AUX port — Router
```

Some Cisco routers have an **AUX (Auxiliary) port**, traditionally connected to a modem for **out-of-band management** — i.e., accessing the router when the normal in-band network path is down or unavailable. It is not used for day-to-day remote administration like SSH/Telnet, but as a backup access path.

Exam One-Liners

- Console and AUX are physical/out-of-band access methods; SSH and Telnet are network/in-band (VTY) access methods.
- Console access is required for the **very first configuration** of a router (no IP/network config exists yet).
- SSH is preferred over Telnet for all remote in-band access.

PART G — Configuration Registers

What Is the Configuration Register?

The **configuration register** is a **16-bit (4 hex-digit) value** stored in **NVRAM** that tells the router **how to boot** — where to look for the IOS, whether to load the startup-config, and other boot-time behaviors.

```
View current value:  
Router# show version  
(look for the line: "Configuration register is 0x XXXX")
```

Common Configuration Register Values

Value	Meaning
0x2102	Default — boot normally: load IOS from Flash, load startup-config from NVRAM
0x2142	Ignore startup-config at boot — router boots into Setup Mode with a blank running-config (commonly used for password recovery)
0x2100	Boot into ROMmon (ROM Monitor) mode — minimal recovery environment, does not load IOS normally

Changing the Configuration Register

```
Router(config)# config-register 0x2142
```

- Takes effect **after the next reload** — not immediately.
- Commonly used during **password recovery**: set to **0x2142** to skip loading the (password-protected) startup-config, log in with no password, then manually reload the old config and change the password, and finally set the register **back to 0x2102** before the final reload/save.

Exam One-Liners

- Config register = 16-bit value in **NVRAM**, controls boot behavior.
 - **0x2102** = normal/default boot.
 - **0x2142** = ignore (skip loading) startup-config — the classic password-recovery value.
 - **0x2100** = boot straight into ROMmon.
 - Changes to the register only take effect **after a reload**.
 - Always remember to set it **back to 0x2102** after password recovery — otherwise the router will keep ignoring its saved config on every future boot.
-

PART H — Cisco IOS Password Recovery (Full Procedure)

If you forget a Cisco router's password, you can recover access using **ROMmon mode**, since physical/console access effectively lets you bypass password protection by controlling the boot process.

Step 1 — Restart and Enter ROMmon

Power off/on the router, and during boot send the **Break** signal to interrupt the normal boot and drop into ROMmon:

```
rommon 1 >
```

Step 2 — Change the Configuration Register

Tell the router to **ignore the startup-config** on the next boot:

```
rommon 1 > confreg 0x2142
```

Why this works:

- **0x2102** → normally loads startup-config (including the password)
- **0x2142** → **ignores** startup-config, so the router boots with no password set

Step 3 — Reset/Reload

```
rommon 2 > reset
```

The router reboots without loading the old (password-protected) configuration.

Step 4 — Enter Privileged Mode

Since the old configuration was skipped, no password is currently applied:

```
Router> enable
Router#
```

Step 5 — Copy the Old Configuration into RAM

The old configuration is still safely stored in NVRAM — it was only skipped, not deleted:

```
Router# copy startup-config running-config
```

```
NVRAM (startup-config)
  | copy
  ▼
RAM (running-config)
```

This restores your old configuration (interfaces, routing, etc.) **without** re-locking you out with the old password.

Step 6 — Change the Password

```
Router# configure terminal
Router(config)# enable secret NewPassword123
```

If needed, also change the console password:

```
Router(config)# line console 0
Router(config-line)# password NewPassword123
Router(config-line)# login
```

Step 7 — Restore Normal Boot Behavior

This step is critical — don't skip it:

```
Router(config)# config-register 0x2102
```

This ensures the router will load the startup-config normally on all future reboots.

Step 8 — Save the Configuration

```
Router# copy running-config startup-config
```

Complete Sequence to Remember

```
Power ON
|
▼
Break
|
▼
ROMmon
|
▼
confreg 0x2142
|
▼
reset
|
▼
Router> enable
|
▼
copy startup-config running-config
|
▼
configure terminal → change password
|
▼
config-register 0x2102
|
▼
copy running-config startup-config
```

★ Exam Shortcut

- **2142 = Ignore startup-config** (skip password)
- **2102 = Normal boot**

2142 → Recover → 2102 → Save.

⚠ Important exam trap: password recovery via ROMmon requires **physical console access** to the router — it cannot be done remotely via Telnet/SSH, since it relies on interrupting the boot process at the console.

PART I — Debugging & Logging

Why Debugging and Logging Matter

Routers and switches constantly generate information about their internal operations, errors, and state changes. **Logging** captures this passively; **debugging** actively surfaces detailed real-time information about specific processes — both are essential for troubleshooting.

Logging (`show logging` / Syslog)

- The router keeps an internal **log buffer** of system messages (interface up/down, config changes, errors, etc.) by default.
- Logs can be sent to multiple destinations simultaneously:

Destination	Command	Notes
Console	(default, always on unless disabled)	Shown directly on the console session
Internal buffer	<code>logging buffered</code>	Stored in RAM; view with <code>show logging</code>
Terminal (VTY sessions)	<code>terminal monitor</code>	Needed to see log messages when connected via Telnet/SSH (not shown by default!)
External Syslog server	<code>logging <syslog-server-ip></code>	Centralizes logs from many devices for long-term storage/analysis

Log Message Severity Levels (0 = most severe → 7 = least severe):

0 Emergency	1 Alert	2 Critical	3 Error
4 Warning	5 Notice	6 Informational	7 Debugging

⚠ **Common gotcha:** if you're remotely connected via Telnet/SSH and don't see log messages that appear on the console, run `terminal monitor` to enable them for your session.

Debugging (`debug` commands)

- `debug` commands provide **real-time, detailed** output about a specific process as it happens (e.g., `debug ip routing`, `debug ip icmp`, `debug spanning-tree events`).
- Extremely useful for diagnosing **active, in-progress** problems (e.g., watching routing updates arrive, or watching an authentication exchange fail step-by-step).

```
Router# debug ip routing
Router# debug ip icmp
Router# undebug all      ← or "u all" — disables ALL active debug output
```

⚠ **Warning:** `debug` commands are **CPU-intensive** — running heavy debugs (like `debug ip packet` on a busy router) can significantly load the CPU and, in extreme cases, cause the router to become unresponsive. Always disable debugging (`undebug all`) as soon as you're done.

Why This Matters — Practical Importance

Without Debugging/Logging	With Debugging/Logging
Problems are invisible until they cause an outage	Early warning signs (interface flaps, failed logins, routing changes) are visible before they escalate
Root-cause analysis after an incident is guesswork	Log history + timestamps let you reconstruct exactly what happened and when
No audit trail of configuration/security events	Logs provide accountability — who did what, and when (especially combined with AAA accounting)
Diagnosing live/intermittent issues is nearly impossible	<code>debug</code> shows the process happening in real time, letting you catch transient issues

Best Practices

- Always timestamp log messages: `service timestamps log datetime msec`
- Send logs to a **centralized syslog server** in production — local buffers are limited in size and lost on reload.
- Use `debug` **sparingly and temporarily** in production — prefer testing in a lab first, and always run `undebug all` when finished.
- Match the **logging severity level** to what's actually needed (`logging console <level>`, `logging buffered <level>`) to avoid being flooded with noise.

Exam One-Liners

- **Logging** = passive record-keeping of system events; **Debugging** = active, real-time, detailed process tracing.
- Logging severity levels run **0 (Emergency)** → **7 (Debugging)** — lower number = more severe.
- `terminal monitor` is required to see log messages over a remote (Telnet/SSH) session.
- `debug` commands are CPU-intensive — always `undebug all` when finished.
- Centralized **Syslog servers** are the production-standard destination for logs (local buffers are volatile and size-limited).